

100

AI Engineering Interview Questions & Answers

The questions behind what actually gets asked in AI engineering interviews, from the most common fundamentals to the senior level follow ups that decide the offer.

MODULE 1 LLMs and RAG

MODULE 2 Agents and orchestration

MODULE 3 Ops and evaluation

MODULE 4 Safety and ethics

MODULE 5 ML foundations

20 questions per module · 100 total

How to use this

These 100 questions were built from what candidates report being asked in real AI engineering interviews, consolidated across interview guides, engineering blogs, and candidate write ups. They are grouped into the five skill areas the Accelerator covers, twenty questions each.

Every question follows the same three part structure so you can practice out loud.

What they are testing. The real reason the question gets asked. Answer this, not just the literal words.

The answer. What a strong response covers, in the order you should say it.

Follow up. The push back that usually comes next. Prepare this and you stay ahead of the interviewer.

Four rules that matter more than memorizing answers

- 1 Always attach a number. Latency, cost per thousand requests, recall at k, escalation rate. Specifics are what separate a shipped system from a tutorial.
- 2 Name the trade off. Knowing what something is scores nothing. Knowing when not to use it is the whole interview.
- 3 Say how you would measure it. Most candidates describe how to build a system and never say how they would know it works.
- 4 Talk about failure modes before you are asked. Production thinking is the signal every interviewer is listening for.

Practice each answer out loud in under two minutes, then let the interviewer pull you deeper. Reading these is preparation. Saying them is practice.

01

MODULE 1

LLMs and RAG

The most asked category in AI engineering interviews. Retrieval design, chunking, grounding, embeddings, and output control.

20 questions

Walk me through a production RAG pipeline end to end, then tell me where quality actually breaks.

What they are testing: *Whether you have shipped one or only read about one.*

The pipeline has eight stages. Ingestion parses source files and preserves structure and metadata. Chunking splits documents with size, overlap, and a link back to the parent section. Embedding turns chunks into vectors with a pinned model version. Indexing stores vectors plus filterable metadata. Retrieval rewrites the query and pulls a wide candidate set. Reranking cuts that set down to the best three to five. Prompt assembly injects context with citation instructions. Generation produces the answer, and post checks validate groundedness and citations before the user sees anything.

Quality almost never breaks where people expect. Ranked by how often I see it: parsing, where tables and multi column PDFs turn into garbage, chunk boundaries that split the answer across two chunks, an embedding model that does not match the domain vocabulary, top k too small so the right chunk never enters the candidate set, no reranking so the answer sits at rank 12, and a stale index. Generation failures are usually a symptom of a weak context, not a weak model.

The debugging method matters more than the list. Measure retrieval in isolation first. If the correct chunk is not in the retrieved set, no prompt change will save the answer. Fix recall at k, then measure answer quality on top of a known good context.

Follow up they will push on: Retrieval recall is high but users still complain. Now you are talking about chunk quality, reranking, and synthesis, not retrieval.

When would you use keyword search, vector search, or both, and why add a reranker on top?

What they are testing: *Trade off fluency. Anyone can say vector database.*

Dense vector search matches meaning, so it handles paraphrase and synonyms. It fails on exact identifiers, rare terms, acronyms, and negation, because the training objective pulls similar sounding text together whether or not the logic matches.

Sparse search such as BM25 matches exact terms, which is what you want for part numbers, error codes, statute references, and names. It has no concept of synonyms.

Hybrid runs both and fuses the results, usually with reciprocal rank fusion or a weighted score. For enterprise corpora that mix prose and identifiers, hybrid is my default rather than an optimization.

Reranking is a second, more expensive pass. A bi encoder embeds the query and the document separately so vectors can be precomputed and searched at scale. A cross encoder reads the query and the document together in one forward pass, which is far more accurate and far too slow to run over millions of documents. So you retrieve 50

to 100 candidates cheaply, then rerank to the top five with a cross encoder. In most systems this is the single largest quality gain per hour of engineering.

The cost is real. Reranking adds roughly 100 to 300 milliseconds and a per document charge, so cap the candidate set and cache aggressively.

Follow up they will push on: How would you pick the embedding model. Answer with your own eval set, not a public leaderboard.

QUESTION 3 · LLMS AND RAG

You are chunking a 300 page report where page 1 says all figures are in thousands. How do you keep that context?

What they are testing: *Whether you understand that chunking destroys document level meaning.*

Fixed size chunking throws away every fact that lives outside the chunk. A number pulled from page 180 is wrong by a factor of a thousand without page 1. There are four fixes and I would use them together.

- ▶ Contextual chunk headers. At ingest time, prepend document title, section path, and key global facts to every chunk. Generate the global facts once per document and reuse them.
- ▶ Parent child retrieval. Embed small chunks for retrieval precision, but pass the parent section to the model for generation, so the answer has surrounding context.
- ▶ Structure aware splitting. Split on headings, sections, and table boundaries rather than character counts. Extract tables separately with their captions instead of flattening them into prose.
- ▶ Metadata filters. Carry document level attributes such as fiscal year, currency, units, and effective date as filterable fields, so the model is not asked to infer them from text.

Operationally, keep overlap modest at roughly 10 to 15 percent, store chunk lineage so citations resolve to a page, and treat re chunking as a versioned reindex job rather than an in place edit.

Follow up they will push on: Contextualizing every chunk costs tokens at ingest. Justify it against the accuracy gain and the size of the corpus.

Retrieval finds nothing relevant and the model answers confidently anyway. How do you fix it?

What they are testing: *Whether you treat hallucination as a system design problem or a prompt problem.*

A model with no context will still produce fluent text, because that is what it was trained to do. The fix is layered and mostly lives outside the prompt.

- ▶ Retrieval confidence gate. Threshold on the reranker score. If the best candidate is below it, do not call the generator with an empty context. Return an explicit no answer plus the closest sources.
- ▶ Prompt contract. Instruct the model to answer only from the provided context, require a citation per claim, and give it an exact refusal string to emit when the context does not support an answer.
- ▶ Groundedness check after generation. Verify each claim against the retrieved text with an entailment model or a judge call, and flag or strip unsupported sentences.
- ▶ Citation validation. Reject any answer whose citations do not resolve to chunk IDs that were actually retrieved. This catches invented sources deterministically and costs nothing.
- ▶ Interface design. Show sources, and treat a clean no answer as a success state rather than a failure.

Then measure it. Abstention rate and unsupported claim rate become production metrics you watch weekly, the same as latency.

Follow up they will push on: How do you stop it refusing too often. Tune the threshold against a labeled set and track false refusals alongside unsupported answers.

Explain temperature and top p, and how you get reliable structured output from a non deterministic model.

What they are testing: *Practical control of model behavior, and whether you know that temperature zero is not determinism.*

Temperature scales the logits before sampling. Low temperature sharpens the distribution toward the highest probability tokens. Top p, or nucleus sampling, truncates the candidate set to the smallest group of tokens whose cumulative probability passes p. Top k truncates to a fixed count. Tune one of them, not all three at once.

In production the rule is simple. Extraction, classification, routing, and tool arguments run near zero. Creative copy and ideation run higher. But temperature zero is not reproducible: batching, floating point non determinism, and silent provider model updates all shift outputs, so never build a test that asserts an exact string.

For structure, stop asking politely for JSON. Use native structured output, JSON schema, or function calling, then validate with a typed model such as Pydantic. On validation failure, retry once with the validation error appended to the context, and cap retries so a bad request cannot loop. Set stop sequences, cap max tokens, and log the schema failure rate as a health metric.

Follow up they will push on: What happens when the provider updates the model under you. Pin versions and gate every change on a regression eval suite.

QUESTION 6 · LLMS AND RAG

How do you choose chunk size and overlap? Give me the reasoning, not a default number.

What they are testing: *Whether you can reason from the retrieval objective instead of copying a tutorial value.*

There is no universal number, because chunk size trades recall against precision. Large chunks carry more context per hit but dilute the embedding, so a chunk about ten topics matches everything weakly. Small chunks embed sharply but strand facts that need their neighbors. The right size is set by the shape of your content and the length of a typical answer span.

Start from the data. Dense reference text, contracts, and documentation tolerate small chunks around 200 to 400 tokens. Narrative or conversational text needs larger spans so a thought is not cut mid sentence. If your answers routinely require stitching several passages, that is a signal to raise size or move to parent child retrieval rather than to keep shrinking.

Overlap exists only to stop a boundary from bisecting an answer, so 10 to 15 percent is usually enough. More overlap inflates index size and returns near duplicate neighbors that crowd out variety in the candidate set.

The honest answer is that you do not guess. Build a labeled retrieval set, sweep two or three chunk configurations, and pick the one with the best recall at k on your own queries. Chunking is an empirical decision you re run when the corpus changes.

Follow up they will push on: Your answers span multiple sections. Now you justify parent child or sliding windows over simply enlarging every chunk.

QUESTION 7 · LLMS AND RAG

What is contextual retrieval, and when is it worth the extra ingest cost?

What they are testing: *Whether you keep up with retrieval technique and can weigh it economically.*

Contextual retrieval prepends a short generated description to each chunk before embedding, explaining where the chunk sits in the document. Instead of embedding a

bare paragraph, you embed that paragraph plus a line such as this section reports Q3 revenue for the EMEA segment. It fixes the core failure of naive chunking, which is that an isolated chunk loses the references, entities, and units that give it meaning.

The mechanism is simple. At ingest you make one cheap model call per chunk to write the context line, then embed the combined text, and ideally index the same text for BM25 so both retrieval paths benefit. Reported results show a meaningful drop in retrieval failures, and pairing it with a reranker compounds the gain.

The cost is one extra generation per chunk at ingest time. Prompt caching over the shared document makes this cheap, and it is a one time cost per document rather than per query, so it amortizes fast on any corpus that gets queried more than a handful of times.

It earns its place when chunks are short, references are heavy, and the corpus is stable enough that reindexing is infrequent. It is poor value on tiny corpora or content that churns hourly.

Follow up they will push on: How is this different from a contextual chunk header you write with a template. One is generated per chunk from the surrounding text, the other is a static document level stamp.

QUESTION 8 · LLMS AND RAG

A user asks a question phrased nothing like the source text. How do query transformation techniques help?

What they are testing: *Whether you know retrieval quality is set before the vector search runs.*

The query the user types is often a poor search key. It may be terse, use different vocabulary than the corpus, or bundle several questions together. Transforming the query before retrieval is frequently a larger win than tuning the index.

- ▶ Query rewriting. Use a cheap model to expand abbreviations, add synonyms, and turn a fragment into a full question, so the embedding lands nearer the source language.
- ▶ HyDE, or hypothetical document embeddings. Ask the model to draft a fake answer, then embed that draft rather than the question. A hypothetical answer looks more like the target passage than the question does, which lifts recall on sparse or mismatched corpora.
- ▶ Decomposition. Split a multi part question into sub questions, retrieve for each, and merge, which is how you handle anything that needs more than one hop.
- ▶ Step back prompting. Ask a broader version of the question first to pull in background, then the specific one.

Each technique adds a model call and latency, so gate them. Route simple lookups straight to retrieval and reserve transformation for queries that come back with low reranker scores or that the router flags as complex.

Follow up they will push on: HyDE can hallucinate a misleading draft. How do you keep a wrong hypothetical from poisoning retrieval. Retrieve on both the raw query and the draft, and let the reranker arbitrate.

QUESTION 9 · LLMS AND RAG

Context windows now hold hundreds of thousands of tokens. Does long context kill RAG?

What they are testing: *Whether you understand cost, latency, and attention behavior, not just headline token counts.*

No, and the reasons are economic and behavioral, not nostalgic. Stuffing a large corpus into every prompt pays the full input token cost on every request, which is the dominant line on most bills, and it raises latency because time to first token grows with input length. RAG pays to retrieve a small relevant slice once, which is far cheaper at any real query volume.

There is also a quality reason. Models attend unevenly across a long context, and relevant facts placed in the middle of a huge window are recalled less reliably than the same facts placed in a tight, curated context. More tokens is not more signal. A focused context of the right five chunks usually beats a hundred marginally related ones.

The two are not rivals. Long context makes RAG better by letting you pass whole parent sections instead of fragments, tolerate looser chunking, and fit richer few shot examples. The pattern is retrieve to narrow, then use the large window to give the model room, not to skip retrieval.

Long context does win for small, bounded inputs, a single contract or one codebase, where retrieval infrastructure is not worth the overhead.

Follow up they will push on: How do you decide the boundary. Estimate tokens per query times price and latency against corpus size, and do the arithmetic out loud.

QUESTION 10 · LLMS AND RAG

Compare fixed size, recursive, semantic, and structure aware chunking. What do you reach for first?

What they are testing: *Depth on the step that quietly determines retrieval quality.*

Fixed size splits on a character or token count with overlap. It is trivial and fast and ignores meaning entirely, so it happily cuts a sentence or a table in half. It is a fine baseline and a poor destination.

Recursive splitting tries a hierarchy of separators, paragraphs, then sentences, then words, falling back only when a piece is too large. It respects natural boundaries far better than a raw counter and is my default starting point for prose.

Semantic chunking embeds sentences and cuts where adjacent sentences diverge in meaning, producing topically coherent chunks. It is more expensive at ingest and can be worth it on dense, mixed content, but it is not automatically better and should be measured, not assumed.

Structure aware splitting uses the document format itself, Markdown headings, HTML tags, PDF layout, code functions, so chunks align with sections and tables stay intact with their captions. For anything with real structure this beats the others, because it preserves the boundaries the author already drew.

In practice I start recursive, add structure awareness for formatted sources, extract tables separately, and only test semantic chunking when evaluation shows boundary problems.

Follow up they will push on: How would you chunk source code. By syntactic unit, function or class, with the file path and imports carried as context.

QUESTION 11 • LLMS AND RAG

What problem does late interaction or multi vector retrieval, such as ColBERT, solve?

What they are testing: *Whether you understand the limits of a single pooled embedding.*

A standard dense retriever squashes a whole passage into one vector. That pooling is lossy: a long or multi topic chunk averages its meaning, and a query that matches one specific phrase can be washed out by everything else in the chunk. This is why single vector search struggles with precise, term level matches inside otherwise unrelated text.

Late interaction keeps a vector per token instead of one per document. At query time it scores every query token against every document token and sums the best matches, so a strong match on a few key terms surfaces the passage even when the rest of it is off topic. It captures fine grained relevance that pooled embeddings lose.

The trade off is storage and complexity. Per token vectors are much larger than one vector per chunk, and the scoring is heavier, so the index costs more to hold and serve. For many teams the simpler and cheaper pattern that reaches similar quality is bi encoder retrieval followed by a cross encoder reranker, which spends the expensive computation only on the shortlist.

So the honest framing is that late interaction targets a real weakness of pooled embeddings, but a reranker often buys most of the same benefit at lower operational cost.

Follow up they will push on: When would you actually pick it over rerank. Very large candidate sets where reranking every one is too slow, and storage is not the constraint.

Your documents are full of tables, charts, and scanned pages. How do you make them retrievable?

What they are testing: *Whether you have handled real documents rather than clean text dumps.*

Naive text extraction is where most enterprise RAG dies. A parser that flattens a table into a run on line destroys the row and column relationships, and a chart or scanned page yields nothing at all. You need to treat non prose content as first class.

- ▶ Layout aware parsing. Use a parser that understands document structure so tables, headers, and columns survive, rather than a plain text dump that reads across columns.
- ▶ Tables as structured objects. Extract each table with its caption and either keep it as Markdown or store it as rows, and generate a short natural language summary of the table to embed alongside it so semantic queries can find it.
- ▶ Images and charts. Run a vision model to produce a text description at ingest and index that description, or move to a multimodal embedding model that can retrieve on the image directly.
- ▶ Scanned pages. OCR first, then flag low confidence output for review, because silent OCR errors become confident wrong answers downstream.

The unifying idea is to convert every modality into something both searchable and faithful at ingest time, and to keep a link back to the original so a citation can show the real table or figure.

Follow up they will push on: A number in a table is retrieved but the model reads the wrong row. What went wrong. Usually table structure was lost in parsing, so fix ingestion, not the prompt.

How do metadata filtering and pre versus post filtering change your vector search?

What they are testing: *Whether you know retrieval is more than nearest neighbor over one flat space.*

Most real queries carry hard constraints that a vector similarity score should never override: this user, this tenant, documents effective this year, this product line. Encoding those as filterable metadata on each chunk and applying them as filters is what keeps retrieval both correct and secure.

Pre filtering restricts the candidate set before the nearest neighbor search, so you only search within the allowed subset. It guarantees the constraint is respected and

is essential for access control, but on some index types a very selective filter can degrade the efficiency of the vector search.

Post filtering runs the vector search first and drops disallowed results afterward. It is simpler and keeps the index fast, but if the filter is selective you can retrieve k results and have almost none survive, so you fetch far more than k or return too little.

The practical answer is to pre filter anything that is a correctness or security requirement, permissions above all, and to lean on post filtering only for soft preferences where a thin result set is acceptable. Either way, permission checks belong at query time on the chunk metadata, never as a hope that the model will decline to repeat what it retrieved.

Follow up they will push on: A selective pre filter makes retrieval slow. What now. Partition or shard by the high cardinality field, or use an index that supports filtered search natively.

QUESTION 14 · LLMS AND RAG

When does a knowledge graph or GraphRAG beat plain vector RAG?

What they are testing: *Whether you match retrieval architecture to the shape of the question.*

Vector RAG retrieves passages that resemble the query. It is excellent for questions answered by one or a few text spans and weak for questions that require connecting facts scattered across many documents, because similarity search has no notion of relationships. Ask how a policy change three hops away affects a specific customer and top k passages will not assemble the chain.

GraphRAG builds an explicit graph of entities and relationships from the corpus, then traverses it to gather connected context, optionally summarizing communities of related nodes. This shines on multi hop reasoning, aggregation across a whole corpus, and questions about how things relate rather than what a passage says.

The cost is heavy. Extracting a reliable graph needs many model calls at ingest, the extraction can be noisy, and maintaining the graph as the corpus changes is real engineering. For the majority of question answering workloads, hybrid retrieval with a reranker is simpler and good enough.

So the decision is driven by the questions, not the hype. If the value is in relationships and multi hop synthesis, a graph earns its keep. If it is in finding the right passage, it is overkill.

Follow up they will push on: Can you get some of the benefit cheaply. Yes, carry structured metadata and links between chunks and let the model follow references, a lightweight middle ground.

How do you keep a RAG index fresh, and what breaks when it goes stale?

What they are testing: *Whether you think about the corpus as living infrastructure, not a one time load.*

A stale index is one of the most common silent failures in production RAG. The system keeps answering fluently from documents that no longer reflect reality, and because the answer looks confident, nobody notices until a user acts on outdated policy or pricing.

The core mechanic is incremental updates keyed on a stable document identifier and a content hash or version. When a source changes, re chunk and re embed only that document and upsert by ID, rather than rebuilding the whole index. Deletions must propagate all the way through, so a removed document leaves the index, the caches, and any derived structures, or it will keep surfacing as a ghost.

Two subtleties bite teams. First, embedding models are versioned, so changing the model means reindexing the entire corpus, since old and new vectors are not comparable. Treat the index as versioned and plan the migration behind a flag. Second, add freshness metadata such as an effective date so retrieval can prefer current documents and the answer can disclose how recent its sources are.

Then monitor it: track index age, update lag, and the rate of no hit queries, because a rising no hit rate often means new content has not been ingested.

Follow up they will push on: Full reindex or rolling migration when you change embedding models. Build a parallel index, validate on your eval set, then cut over, never edit in place.

Walk me through prompt caching and how you construct context to exploit it.

What they are testing: *Whether you can cut cost and latency with the structure of the prompt itself.*

Prompt caching lets a provider store the processed representation of a prefix so repeated requests that share that prefix skip most of the input computation. The result is lower latency and a large discount on the cached tokens. The catch is that caching is prefix based, so it only helps if the shared content sits at the front and is byte identical across calls.

That dictates how you order a prompt. Put the stable material first, the system instructions, tool definitions, and any long reference text that does not change, then the variable material, the retrieved chunks and the user question, last. If you interleave a per request timestamp or a user name into the prefix, you break the cache for everyone.

It pays off most when many requests share a large fixed context: a big system prompt, a fixed knowledge base passage, or a long set of few shot examples. In agent loops it is especially valuable, because each step resends a growing history and the stable head can stay cached across steps.

Measure the hit rate. A low cache hit rate usually means something dynamic leaked into the prefix, and fixing the ordering is often the cheapest latency and cost win available.

Follow up they will push on: Caching has a short time to live. How do you keep a hot prefix warm. Keep traffic flowing to it or accept the first call in a burst pays full price.

QUESTION 17 • LLMS AND RAG

Two retrieved documents contradict each other. How should the system behave?

What they are testing: *Whether you design for the messy reality that corpora disagree.*

Real corpora contradict themselves constantly: an old policy and its revision, two regions with different rules, a draft and a final. A naive pipeline retrieves both, and the model silently picks one, usually the one that appears first, which is not a decision anyone chose.

The first fix is metadata so the system can tell versions apart: effective dates, document status, source authority, and region. With that, retrieval can prefer the current or authoritative version, and a reranker can weight it up. Much apparent contradiction is really a freshness or scope problem that metadata resolves before generation.

When a genuine conflict remains, do not paper over it. Instruct the model to surface the disagreement, attribute each claim to its source, and note the conditions under which each holds, rather than asserting one as fact. For high stakes domains a clean I found conflicting sources, here is each, with citations is the correct and safer answer.

Then treat recurring contradictions as a data quality signal. If the same conflict keeps appearing, the fix is upstream, deduplicate, retire the stale document, or add scope metadata, not another prompt patch.

Follow up they will push on: The model still merges two conflicting numbers into one wrong figure. Why. Contradictory chunks in one context invite blending, so detect the conflict at retrieval and separate or gate before generation.

How do you evaluate retrieval specifically, and how do you build the labeled set to do it?

What they are testing: *Whether you can isolate the retrieval layer instead of judging only the final answer.*

You cannot fix what you measure only end to end, because a bad final answer could be retrieval or generation and the single score does not tell you which. So evaluate retrieval on its own first, against a set of queries each mapped to the documents that should have been found.

- ▶ Recall at k. Of the documents that should appear in the top k, how many do. This is the metric that matters most, because a chunk that is never retrieved can never be used.
- ▶ Precision at k. How much of the top k is actually relevant, which speaks to how much noise the generator must filter.
- ▶ MRR. How high the first relevant result ranks, which matters when the model weights early context more.
- ▶ NDCG. A rank aware score that rewards putting the most relevant results first when relevance is graded rather than binary.

Building the set is the real work. Sample real queries from production logs, stratified across common, long tail, ambiguous, and out of scope, then label each with the source documents that should answer it. You can bootstrap labels by having a strong model propose relevant chunks and a human confirm, which is faster than labeling from scratch. Version the set and add every production failure to it, so each incident becomes a permanent regression test.

Follow up they will push on: Retrieval scores are strong but answers are still wrong. Where do you look next. Chunk quality, reranking, and the generation step on top of a known good context.

What is reciprocal rank fusion, and why is it a sensible default for combining rankers?

What they are testing: *Whether you understand how to merge results from systems with incomparable scores.*

When you run vector search and BM25 together, you face a merging problem: their scores live on different scales, a cosine similarity and a BM25 relevance number are not comparable, so you cannot simply add them. Normalizing scores is fragile because the distributions shift with every query.

Reciprocal rank fusion sidesteps this by ignoring the raw scores and using only the rank position. Each document gets a contribution of one over a small constant plus its

rank in each list, and those contributions are summed across rankers. A document that ranks high in either system floats to the top, and one that ranks decently in both is rewarded. Because it depends only on ordering, it is robust to the scale mismatch that breaks weighted score fusion.

It is a strong default precisely because it needs almost no tuning, just the one smoothing constant, and it degrades gracefully. When you have a labeled set and want to squeeze out more, a learned weighting or a reranker over the fused set will usually beat plain fusion, but fusion is the reliable baseline you reach for first.

The practical note is to fuse a wide enough candidate set from each ranker, then rerank the fused shortlist, which combines robust merging with precise final ordering.

Follow up they will push on: When does simple fusion underperform. When one retriever is far stronger for your domain, in which case weight it or let a reranker sort the merged pool.

QUESTION 20 • LLMS AND RAG

In a multi turn chat, the user asks a follow up like what about the second one. How do you retrieve for a question that only makes sense given the history?

What they are testing: *Whether you handle retrieval inside a conversation, where the query is incomplete without prior turns.*

Retrieval quietly assumes the query is self contained, but in a real chat it rarely is. A turn like what about the second one, or can it handle refunds, carries pronouns and ellipsis that point back at earlier messages. Embed that raw string and you retrieve noise, because the words on their own do not describe what the user means.

The standard fix is query contextualization, sometimes called history aware retrieval. Before searching, you have a cheap model rewrite the latest turn into a standalone question using the recent history, so what about the second one becomes what are the fees for the premium plan. You embed and search on that rewritten query, not the literal follow up. This one extra call is far cheaper than the wrong answer it prevents.

- ▶ Rewrite, do not dump. Feeding the whole transcript into the embedder dilutes the signal. A single clean rephrased question retrieves better than ten turns of concatenated chat.
- ▶ Bound the history. Use the last few turns, not the entire conversation, or the rewrite drifts and latency grows.
- ▶ Detect topic switches. When the new turn is already self contained or changes subject, skip the rewrite so you do not drag stale context into a fresh question.

Measure it the same way as any retrieval, recall at k, but build the eval set from real multi turn logs where the final turn is context dependent, since those are exactly the cases a single turn benchmark will never catch. The failure mode to watch

is a rewrite that invents specifics the user never said, which sends retrieval confidently in the wrong direction.

Follow up they will push on: The rewrite model adds a detail the user did not mention. How do you contain that. Keep the rewrite extractive and grounded in the visible turns, and log rewrites so you can catch drift.

02

MODULE 2

Agents and orchestration

Where senior candidates separate themselves. Control flow, tool design, memory, routing, and knowing when not to use an agent.

20 questions

What is the difference between an agent and a chain of LLM calls, and when is an agent the wrong choice?

What they are testing: *Whether you reach for agents because the problem needs them or because they are exciting.*

A chain is a control flow you wrote. The steps, the order, and the exit are fixed. An agent decides at runtime which tool to call, how many steps to take, and when it is finished. The dividing line is dynamic control flow driven by model output.

An agent earns its cost when the path cannot be enumerated in advance, when input variability is high, and when the number of steps depends on what the system discovers along the way. Research, triage across messy sources, and open ended debugging fit.

An agent is the wrong choice when the workflow is already known, because a pipeline is cheaper, faster, testable, and debuggable. It is also wrong under a tight latency budget, when errors are expensive and hard to reverse, and when you cannot write down a clear success test. Most production systems marketed as agents are workflows with one or two model driven decision points, and that is usually the correct architecture.

The senior answer: start deterministic, and add autonomy only at the points where measured failures justify it.

Follow up they will push on: One agent with more tools, or several specialized agents. Multi agent buys context separation and specialization at the cost of latency, tokens, and coordination bugs, so justify it with a measured win.

Walk me through a production agent architecture. What exists besides the model?

What they are testing: *Whether you have run one in production or only built a demo.*

- ▶ Orchestrator. Owns control flow, retries, budgets, and termination. This is code, not prompt text.
- ▶ Tool layer. Typed schemas, scoped credentials, timeouts, idempotency keys, and structured errors the model can reason about.
- ▶ State store. Durable run state so a long job survives a restart or a deploy, and so you can resume rather than replay.
- ▶ Memory. A small curated working context plus a long term store that is written through an explicit extraction step.
- ▶ Planning and reflection. Only where it earns its tokens. A plan then execute split beats re planning at every step for most workloads.
- ▶ Termination conditions. Step budget, token budget, wall clock, cost ceiling, and an explicit success test.
- ▶ Human in the loop gates. Required for irreversible or externally visible actions.

- ▶ Observability. Per step traces with inputs, outputs, tool results, tokens, latency, and cost.
- ▶ Eval harness. Recorded runs replayed against every prompt, schema, or model change.

The organizing principle is that the model proposes and the orchestrator disposes. Putting control flow in the prompt is the most common architectural mistake in agent systems, because it makes behavior untestable and unversionable.

Follow up they will push on: How do you version and roll back agent behavior. Prompts, tool schemas, and model versions are all deploy artifacts behind flags, gated by trace replay.

QUESTION 23 · AGENTS AND ORCHESTRATION

How does an agent decide which tool to call, and how do you reduce hallucinated tool calls?

What they are testing: *Whether you understand tool selection as a design problem you control.*

Tool selection is driven entirely by what is in context: the tool names, the schema, the descriptions, and any examples. It is a prompting and retrieval problem, so it is yours to fix.

- ▶ Keep the tool set small and non overlapping. Thirty tools with fuzzy boundaries produce fuzzy selection. Merge or remove before you tune prompts.
- ▶ Write descriptions that say when not to use the tool, not just what it does. Negative guidance cuts misfires sharply.
- ▶ Use strict typed parameters, enums instead of free text, required fields, and short examples of valid arguments.
- ▶ Validate arguments before execution and return a structured error the model can correct, rather than throwing a stack trace into the context.
- ▶ If the catalog is large, retrieve the relevant subset of tools per step instead of listing everything.
- ▶ Always give it a way out. A none of these option prevents forced selection of the least bad tool.

On reliability, every call needs a timeout, a retry policy with backoff, and an idempotency key so a retry does not charge a customer twice. Distinguish retryable failures from terminal ones, and pass terminal ones back to the model as facts about the world.

Follow up they will push on: How do you measure this. Tool selection accuracy and argument validity rate against a labeled set of tasks.

Your agent loops on the same tool and burns budget. How do you detect and stop it?

What they are testing: *Production instincts. This is the most common real agent failure.*

Hard limits first, enforced by the orchestrator rather than requested in the prompt: max steps, max tokens per run, max wall clock, and max cost per run and per tenant.

Then loop detection. Hash the tool name plus normalized arguments on every call. If the same hash repeats, block the call and return the earlier result along with a note that it was already attempted, which forces a different branch. Track progress separately: if several steps pass without new information entering state, terminate and return the best partial result.

For over planning, cap planning depth and do not allow a full re plan on every step by default. Route routine steps to a cheaper model so exploration is affordable.

Failure should be graceful. Return the best partial answer with a stated reason, never a raw exception and never an endless spinner. Alert on p99 steps per run, because a rising tail is the early signal of a prompt or schema regression.

Follow up they will push on: Which agent failure is hardest to catch. Silent success, where the agent skips a required step and returns a confident answer. Only step level traces plus outcome evals surface it.

How do you design agent memory, and when do you insert a human?

What they are testing: *Whether you can keep long running systems coherent without polluting them.*

Four kinds of memory are worth naming. Working memory is the live context for the current run. Episodic memory is what happened in past runs and conversations. Semantic memory holds durable facts about the user and domain. Procedural memory is the routines the system knows, which in practice is your prompts and tools.

The design rule is that the live context stays small and curated. Summarize older turns, keep the raw transcript in storage, and retrieve into context only what the current step needs. Write to long term memory through an explicit extraction step with a schema, never by dumping transcripts, because raw dumps carry speculation, corrections, and errors that later get retrieved as fact.

Every memory record carries a source, a timestamp, and a confidence. Retrieval filters on recency and relevance. Contradictions resolve newest first, with older records retained for audit rather than deleted silently.

For human in the loop, gate on irreversibility and blast radius, not on model confidence. Money movement, data deletion, outbound messages, and anything a customer sees needs approval. Show the reviewer a diff of the proposed action rather than a

wall of reasoning, capture the decision, and feed approvals and rejections back into your eval set so the gate gets narrower over time.

Follow up they will push on: How do you keep memory from growing unbounded. Expiry policies, deduplication at write time, and a size budget per user.

QUESTION 26 · AGENTS AND ORCHESTRATION

Compare ReAct, plan and execute, and reflection. When does each pattern fit?

What they are testing: *Whether you know the common agent control patterns and their costs, not just the buzzwords.*

ReAct interleaves reasoning and acting: the model thinks, calls a tool, reads the result, and repeats. It adapts well when each step depends on what the last one returned, which suits exploration and debugging, but it makes a fresh decision every step, so it drifts on long tasks and spends tokens reasoning at each turn.

Plan and execute splits the work: the model writes a plan up front, then executes the steps, replanning only when a step fails. This is cheaper and more predictable on tasks whose shape is roughly knowable, and it keeps the agent from wandering, at the cost of being less reactive when reality diverges from the plan.

Reflection adds a critique pass: the agent evaluates its own output against the goal and revises. It lifts quality on tasks with a checkable target, code that must pass tests, an answer that must satisfy constraints, but it multiplies calls and can spiral if the critique is vague, so it needs a hard iteration cap.

The practical answer is that these compose. A common production shape is plan and execute for structure, tool use within each step, and a bounded reflection loop only where output correctness can be verified. Start with the least autonomy that works and add loops where measured failures justify them.

Follow up they will push on: Reflection keeps improving on paper but not in reality.

Why. The critic is grading against its own reasoning, not an external check, so ground the critique in tests or retrieved evidence.

QUESTION 27 · AGENTS AND ORCHESTRATION

When is multi agent worth it, and what breaks when you split one agent into many?

What they are testing: *Whether you can justify coordination overhead instead of adopting a fashionable pattern.*

Multi agent means several specialized agents, often a supervisor delegating to workers, or agents that hand off to one another. The real benefits are context isolation, each agent keeps a smaller, cleaner context focused on its job, and specialization, each can have its own tools, prompt, and even model.

Those benefits are concrete when the task has genuinely separable subdomains, when a single context would overflow or get muddled, or when parallelism across independent subtasks saves wall clock time. Broad research that fans out into independent threads is a good fit.

What breaks is coordination. Every handoff is a place to lose information, latency and token cost multiply because agents talk to each other as well as to tools, errors compound across boundaries, and debugging gets harder because a failure may live in the seam between agents rather than in any one of them. Shared state and consistency become their own problem.

So the bar is high. Most tasks that look like they need a team of agents are better as one well scoped agent, or a workflow with a couple of specialized steps. Reach for multi agent when isolation or parallelism delivers a measured win that outweighs the coordination tax, not by default.

Follow up they will push on: How do agents share state without corrupting each other. A durable shared store with clear ownership and versioning, not by passing ballooning message histories around.

QUESTION 28 · AGENTS AND ORCHESTRATION

How do you design tool schemas and descriptions so the model uses them reliably?

What they are testing: *Whether you treat the tool interface as the surface that determines agent behavior.*

The model only knows about a tool what the schema and description tell it, so the interface is the single biggest lever on tool use quality. Vague names and loose types produce vague, malformed calls.

- ▶ Name and describe by intent. State what the tool is for and, crucially, when not to use it, so the model can disambiguate against neighbors.
- ▶ Constrain the types. Use enums instead of free strings, mark required fields, set formats and ranges, so the space of invalid calls shrinks before the model even chooses.
- ▶ Keep parameters flat and minimal. Deeply nested or optional heavy schemas invite errors, so expose the smallest set of arguments that does the job.
- ▶ Show one or two examples of a correct call, which anchors format far better than prose.
- ▶ Return structured, actionable results and errors. On a bad argument, return what was wrong and how to fix it, not a stack trace, so the model can self correct.

Design the granularity too. A few well scoped tools beat many overlapping ones, because overlap creates ambiguity. And if the catalog is large, retrieve a relevant subset per step so the model is not choosing among dozens at once.

Follow up they will push on: How do you know your descriptions are good. Measure tool selection accuracy and argument validity on a labeled task set, then rewrite the descriptions that misfire.

QUESTION 29 · AGENTS AND ORCHESTRATION

A long agent run overflows the context window. How do you manage context over many steps?

What they are testing: *Whether you can keep a long task coherent without exhausting the window or the budget.*

Every step appends the model output and the tool result, so a long run grows without bound, and eventually you either overflow the window or pay for a huge context on every remaining step. Context management is the difference between an agent that finishes and one that degrades.

- ▶ **Compaction.** Periodically summarize older steps into a compact running state, keeping the goal, key findings, and open subtasks, while the raw history moves to storage you can retrieve from if needed.
- ▶ **Externalize state.** Hold the task list, intermediate artifacts, and results in a state store rather than in the prompt, and pull only what the current step needs back into context.
- ▶ **Selective inclusion.** Do not resend every tool result verbatim. Keep the outputs that matter, reference the rest by ID, and let the agent fetch details on demand.
- ▶ **Scratchpad discipline.** Give the agent an explicit place to write notes and decisions, so its working context is structured rather than a growing transcript.

The principle is that the live context should hold what the next decision requires, not the entire history. Storage is cheap and the window is not, so push durable state out and keep the prompt lean.

Follow up they will push on: Summarizing loses a detail the agent later needs. How do you prevent that. Keep the raw history retrievable and let the agent search it, so compaction is lossy in the prompt but not in the system.

QUESTION 30 · AGENTS AND ORCHESTRATION

What is the Model Context Protocol, and why does a standard for tools matter?

What they are testing: *Whether you follow how the ecosystem is standardizing agent tooling.*

The Model Context Protocol is an open standard for connecting models to tools and data sources through a common interface. Instead of writing a bespoke integration for every model and every tool combination, a tool is exposed once as an MCP server and any MCP

compatible client can use it. It standardizes how tools, resources, and prompts are described and invoked.

The problem it solves is combinatorial. Without a standard, every application reimplements connectors for each data source, and each new model or framework means redoing that work. A shared protocol turns an N times M integration problem into N plus M : build a server once, and many clients can consume it.

For an engineer this changes the build versus reuse calculus. You can adopt existing servers for common systems, wikis, ticketing, databases, and expose your own internal capabilities as a server that any agent in the company can call, with the description and schema living in one place.

The cautions are the usual ones for any tool surface. An MCP server is still an execution boundary, so it needs scoped credentials, permission checks, and validation, and untrusted content returned by a server is still a potential injection vector. A standard makes wiring easier, not safety automatic.

Follow up they will push on: Does MCP remove the need for tool design. No, the schema and description quality still drive selection, and access control is still yours to enforce at the server.

QUESTION 31 • AGENTS AND ORCHESTRATION

How do you handle errors and retries in agent tool calls?

What they are testing: *Whether you build for a world where tools fail, time out, and return junk.*

Tools live in the real world, so they fail: timeouts, rate limits, malformed responses, and genuine business errors. An agent that treats every failure the same either gives up too early or hammers a dying dependency. The first move is to classify failures.

Transient failures, timeouts, rate limits, brief outages, should be retried automatically by the orchestrator with exponential backoff and a cap, without ever involving the model. Terminal failures, invalid input, not found, permission denied, should not be retried blindly. Instead, return them to the model as structured facts about the world so it can choose a different path, because retrying a genuine not found just wastes steps.

Idempotency is essential for anything with side effects. Attach an idempotency key so that a retry of a payment or a message does not execute twice, and design write tools to be safe under repetition. This is the difference between a robust agent and one that double charges a customer on a flaky network.

Finally, cap total retries across the run, degrade gracefully to a partial result with a stated reason, and log every failure with its classification so you can see which tools are unreliable and fix them upstream.

Follow up they will push on: A tool returns success but with garbage data. How do you catch it. Validate tool outputs against a schema and sanity checks, not just HTTP status, because a 200 with nonsense is still a failure.

QUESTION 32 · AGENTS AND ORCHESTRATION

How do you evaluate an agent, given that it can succeed by different paths?

What they are testing: *Whether you can measure multi step systems where the trajectory varies.*

A single LLM call is judged on its output. An agent is harder because two runs can reach the same correct answer by different paths, or reach a wrong answer through plausible looking steps, so you have to evaluate both the outcome and the trajectory.

- ▶ Outcome evaluation. Did the run achieve the goal, measured against a task success criterion or a checkable end state, not against a fixed expected transcript.
- ▶ Trajectory evaluation. Did it use the right tools, avoid unnecessary steps, and stay within budget. Step level traces let you score tool selection accuracy and efficiency, and catch silent success where a required step was skipped.
- ▶ Component evaluation. Test the pieces in isolation too, tool argument validity, routing decisions, and any retrieval, so a failure can be localized.

The backbone is recorded runs. Capture real trajectories with inputs, tool calls, and outputs, curate a set of representative and failure cases, and replay them against every prompt, schema, or model change as a regression suite. Because agents are non deterministic, run key cases multiple times and look at success rate rather than a single pass.

Follow up they will push on: The agent passes your fixed cases but fails in production. Why. Your set under samples real input variety, so mine failures from logs continuously and add them back.

QUESTION 33 · AGENTS AND ORCHESTRATION

When should an agent run tools in parallel, and what makes that safe?

What they are testing: *Whether you can exploit concurrency without creating races and inconsistent state.*

Many agent tasks contain independent subtasks, fetching from three sources, checking several records, that a strictly sequential loop runs one at a time, wasting wall clock. Running independent tool calls concurrently is often the single biggest latency win in an agentic system.

The precondition is independence. Parallelize only calls that do not depend on each other output and do not write to the same state, which is why read heavy fan out is

the natural fit. Steps with a data dependency must stay ordered, and mixing the two requires the orchestrator to track which results feed which next call.

Safety comes from the orchestrator, not the model. It should bound concurrency to respect rate limits and cost, aggregate results deterministically once all branches return, and handle partial failure explicitly, decide whether a missing branch fails the step or degrades the answer. Writes need special care: concurrent writes to shared state invite races, so either serialize them or make them idempotent and conflict aware.

So the rule is simple. Fan out reads freely under a concurrency cap, keep dependent and write steps ordered, and let code, not the prompt, coordinate the joins.

Follow up they will push on: Two parallel branches both try to update the same record. What happens. A race, so serialize writes or use optimistic concurrency with versioning, never assume ordering.

QUESTION 34 · AGENTS AND ORCHESTRATION

How do you make a long running agent durable across restarts and deploys?

What they are testing: *Whether you treat an agent run as a long lived process, not a single request.*

A task that takes minutes or hours will inevitably be interrupted by a deploy, a crash, or a timeout. If the entire state lives in memory, the interruption throws away all the work and, worse, may re execute side effects when the run restarts from scratch. Durability is what makes long tasks viable.

The core pattern is to persist run state at each step to a durable store: the current step, the plan, intermediate results, and which side effecting actions have already completed. On restart, the orchestrator loads that state and resumes from the last good point rather than replaying from the beginning. This is the same idea as a durable workflow engine, and using one is often the right call.

Side effects are the sharp edge. Record that a payment or an email has been sent before or atomically with performing it, and key those actions with idempotency tokens, so a resume does not repeat them. Replaying reads is harmless, replaying writes is not.

Finally, make steps checkpointable and reasonably granular, so a resume loses at most one step of work, and expose run status so a stuck or long run is observable and can be inspected or cancelled.

Follow up they will push on: How do you avoid double executing an action after a crash mid step. Persist an intent record and an idempotency key before the call, and check it on resume, so the action runs exactly once.

How do you budget and control cost in an agentic system?

What they are testing: *Whether you can keep an autonomous loop from quietly spending unbounded money.*

An agent decides its own number of steps, so without limits a single hard task, or a loop, can consume far more tokens than any human request would, and multiplied across tenants that is a runaway bill. Cost control has to be structural, enforced by the orchestrator.

- ▶ Hard ceilings per run and per tenant. A max step count, a max token budget, and a max spend, checked before each call, so a run stops rather than spirals.
- ▶ Model tiering. Route routine steps, classification, extraction, simple reasoning, to a cheaper model, and reserve the frontier model for the hard steps, tracking the escalation rate as a dial.
- ▶ Context discipline. Compaction and selective inclusion keep the per step context small, since input tokens dominate cost in a growing loop, and prompt caching discounts the stable prefix.
- ▶ Loop and progress guards. Detect repeated calls and stalled progress and terminate, because most cost blowups are a failure to make progress, not legitimate work.

Then attribute it. Log tokens and cost per step, per run, per tenant, and per feature, so you can see where money goes, set alerts, and optimize the expensive paths rather than guessing.

Follow up they will push on: One tenant is far more expensive than the rest. What do you do. Look at their trajectories, they are likely hitting a hard case that loops, then fix the failure mode and cap that tenant.

How do you route a request to the right model, tool, or agent?

What they are testing: *Whether you can build the dispatch layer that makes a system efficient and accurate.*

Routing is the decision, made before the main work, of where a request should go: which model handles it, which tool or agent owns it, or whether it needs the full pipeline at all. Done well it cuts cost and latency and improves accuracy by sending each request to the component best suited to it.

The simplest and often sufficient router is a small, fast classifier, a cheap model or even a lightweight trained model, that labels intent or difficulty and dispatches accordingly. Easy factual lookups go to a small model, complex reasoning to a frontier model, and out of scope requests are declined early. For tool or agent selection, the same idea picks the right specialist.

The trade off is that the router itself is a component that can be wrong, and a misroute sends a request down the wrong path entirely. So keep the routing decision measurable, evaluate routing accuracy on a labeled set, provide a fallback or escalation path when confidence is low, and design so a borderline case fails toward the more capable option rather than the cheaper one.

Keep the router simple. An over clever router is hard to debug and drifts, whereas a small classifier with a clear fallback is easy to reason about and easy to fix.

Follow up they will push on: The router sends hard queries to the cheap model and quality drops. How do you fix it. Add a confidence based escalation so low confidence answers are retried on the stronger model, and retune the threshold on labeled data.

QUESTION 37 · AGENTS AND ORCHESTRATION

How do you design human handoff and escalation in an agent workflow?

What they are testing: *Whether you can build a graceful boundary between automation and people.*

Automation should escalate on purpose, not fail into a dead end. The two questions are when to hand off and how to hand off so the human can act quickly.

On when, escalate on irreversibility and blast radius first, money movement, data deletion, anything a customer sees, which get approval regardless of model confidence. Escalate also on low confidence, on repeated failure or looping, and on explicit user request or detected frustration. Encode these as deterministic gates in the orchestrator, not as a hope that the model will ask for help.

On how, hand the human a decision, not a puzzle. Show the proposed action as a concrete diff, the relevant context, and a recommended option, so the review is a quick yes, no, or edit rather than a reconstruction of what the agent was doing. Preserve the run state so the agent can resume from the human decision without redoing work.

Then close the loop. Capture every approval, rejection, and edit and feed it back into the eval set and the gating rules, so the system learns which cases genuinely need a human and the gate narrows over time instead of escalating everything forever.

Follow up they will push on: You escalate too much and overwhelm reviewers. What now. Tighten the gates using the logged decisions, automate the classes humans always approve, and reserve review for genuine risk.

What guardrails are specific to agents that take actions, beyond text filtering?

What they are testing: *Whether you understand that an acting system needs controls a chat model does not.*

A chat model can say something wrong. An acting agent can do something wrong, delete data, send a message, spend money, so its guardrails have to govern actions, not just words.

- ▶ Least privilege on tools. Scope every credential to the requesting user permissions, separate read tools from write tools, and never let retrieved content or a tool result expand what the agent is allowed to do.
- ▶ Action gating. Route irreversible or external actions through a deterministic policy check or human approval that the model cannot talk its way past, so the decision to act is enforced in code.
- ▶ Argument validation and allowlists. Validate every tool argument before execution and constrain destinations, recipients, tables, and amounts to allowlists, so a wrong argument cannot reach production.
- ▶ Sandboxing and budgets. Execute risky work in isolation, and give every tenant an action budget and a kill switch, so blast radius is bounded even when something goes wrong.

The unifying principle is to design for containment: assume the model will eventually propose a harmful action and make sure the system, not the prompt, prevents it from happening. Then log every action with its inputs and effects so an incident can be reconstructed.

Follow up they will push on: Injection in a retrieved document tells the agent to email data out. What stops it. Channel separation plus output and action gating, untrusted content is data, and the exfiltration action fails the policy check regardless of what the text says.

How does an agent decompose a complex task, and how do you keep decomposition from going wrong?

What they are testing: *Whether you understand planning as a controllable, checkable step rather than magic.*

Decomposition is the agent turning a broad goal into an ordered set of smaller, executable subtasks. It matters because a single leap to the answer fails on anything non trivial, whereas a good breakdown makes each step tractable, checkable, and often parallelizable.

In practice you have the model produce an explicit plan, a list of subtasks with dependencies, before execution, then execute against it and replan only when a step fails or new information invalidates the plan. Keeping the plan explicit and external, rather than implicit in a running monologue, means you can inspect it, resume from it, and evaluate it.

The failure modes are specific and worth naming. Plans can be too coarse, one subtask that is still the whole hard problem, or too fine, dozens of trivial steps that waste calls. They can miss dependencies, run a step before its input exists, or be unverifiable, no way to tell whether a subtask actually succeeded. Guard against these by requiring each subtask to have a clear done condition, capping planning depth, and checking dependencies before execution.

The strongest sign of a mature answer is grounding the plan in reality: verify each step outcome before moving on, so errors are caught locally instead of compounding to the end.

Follow up they will push on: The plan looks good but a mid step failure corrupts everything after it. Why. No verification between steps, so a wrong intermediate result propagates, add a check and replan on failure rather than trusting the plan blindly.

QUESTION 40 · AGENTS AND ORCHESTRATION

A user is watching an agent work and realizes it is heading the wrong way. How do you let them interrupt, correct, or steer it mid task?

What they are testing: *Whether you design agents a human can control while they run, not just start and wait on.*

An agent that cannot be interrupted is a liability, because the moment a user sees it misread the goal, their only options are to let it finish the wrong task or kill it and lose all progress. Steerability is an architecture decision, not a prompt, and it has to be built into the loop from the start.

The foundation is that the agent runs as an interruptible loop with checkpointed state after each step, rather than one long blocking call. Between steps you check for a pending user message, and if one arrives you fold it into the context and let it change the next decision. Because state is checkpointed, a correction redirects the run instead of restarting it, which is the difference between steering and starting over.

- ▶ Stream the reasoning and actions so the user can see where it is going in time to intervene, not after the fact.
- ▶ Gate irreversible actions behind confirmation, so a wrong turn pauses at the send email or place order step rather than committing.
- ▶ Accept mid run input as first class, injected as a new instruction that outranks the original plan when the two conflict.

The trade off is latency and complexity, since checkpointing and confirmation add round trips, so reserve the heavier controls for consequential or long running tasks and let cheap read only steps flow. Measure it by how often users abandon or hard kill runs, and by whether a correction actually changes the outcome. The failure mode is an agent that acknowledges the correction in text but keeps executing the original plan, which means the new instruction never reached the part of the loop that chooses actions.

Follow up they will push on: The agent says understood, correcting now, then does the original thing anyway. Why. The correction was added to the transcript but not given precedence in planning, so the plan step still reads the stale goal.

03

MODULE 3

Ops and evaluation

The round candidates most often say they wish they had prepared for. Metrics, judges, monitoring, rollout, reliability, and unit economics.

20 questions

How do you evaluate a RAG chatbot? Be specific about metrics.

What they are testing: *The single most common question candidates fail. Vibes are not an answer.*

Evaluate each layer separately, then end to end, because a single end to end score tells you nothing about what to fix.

- ▶ Retrieval layer. Recall at k, precision at k, MRR, and NDCG against a labeled set of queries mapped to the documents that should have been found.
- ▶ Generation layer, given a known good context. Groundedness or faithfulness, meaning every claim traces to the context. Answer relevance, completeness, and citation correctness.
- ▶ End to end. Task success as the business defines it, plus safety pass rate and refusal correctness.

The golden dataset is the real work. Sample 100 to 300 queries from production logs, stratified across head queries, long tail, ambiguous phrasing, adversarial input, and out of scope requests. Label each with the expected source documents and either an acceptable answer or a rubric. Version it, and add a case every time a bug is found, so every incident becomes a permanent regression test.

Name the pitfalls too. BLEU and ROUGE measure n gram overlap and correlate poorly with correctness on open ended answers. Perplexity measures how well a model predicts text, not whether the task succeeded.

Follow up they will push on: How do you keep the eval honest. Hold out a slice that whoever tunes the prompts never sees.

You are using an LLM as a judge. How do you know the judge is trustworthy?

What they are testing: *Whether you treat automated evaluation as a measurement instrument that itself needs calibration.*

Judges are useful and biased. The job is to make the bias measurable rather than to pretend it is absent.

- ▶ Write a concrete rubric with a short ordinal scale, three or five points, each point defined. Rate one to ten produces noise.
- ▶ Give the judge the reference material and require a written reason before the score, which improves consistency.
- ▶ Build a calibration set of 100 to 300 human labeled examples and measure agreement between judge and human with Cohen kappa or correlation. Iterate the rubric until agreement is acceptable, and report the number when you present results.

Known biases to control for: position bias in pairwise comparisons, fixed by randomizing order and running both directions; length bias, where longer answers score higher; self preference, where a model favors output from its own family, fixed by judging with a different model; and leniency drift when the rubric is vague.

Use pairwise comparison for ship or no ship decisions between two versions, and absolute scoring for regression thresholds over time. Judging is inference, so sample traffic rather than scoring everything, batch the calls, and pin the judge model version or your scores stop being comparable across weeks.

Follow up they will push on: When do you still need humans. Anything the judge disagrees with itself on, plus a rotating sample for drift detection.

QUESTION 43 · OPS AND EVALUATION

What do you monitor once an AI feature is live, and how do you diagnose a quality drop?

What they are testing: *Production thinking. Interviewers want to hear that you watch quality, not just uptime.*

Log every request as a trace: input, retrieved chunk IDs, prompt version, model and version, tool calls, output, tokens, cost, per stage latency, and any user feedback. Without traces the rest is guesswork.

- ▶ System metrics. p50, p95, and p99 latency, time to first token, error and timeout rate per provider, token spend per feature and per tenant.
- ▶ Quality metrics on sampled traffic. Groundedness scores, citation resolution rate, refusal rate, schema validation failures, and retrieval no hit rate.
- ▶ Business metrics. Task completion, containment or deflection rate, escalation rate, negative feedback rate, and repeat question rate. These are the numbers a hiring manager actually reports upward.

When accuracy drops, do not retrain first. Work the dependency chain: did the index go stale or partially fail to update, did an upstream parser change, was a prompt or model version deployed, did a new traffic segment arrive with different vocabulary or language, did a provider change default behavior. Compare the failing cohort against a stable one before touching the model.

Follow up they will push on: What do you page on versus review weekly. Page on cost, latency, and error rate. Review quality trends on a schedule.

How do you roll out a new model or a new prompt without breaking users?

What they are testing: *Release discipline, which is where software engineering experience transfers directly.*

- ▶ Offline gate. Run the golden set. No quality regression beyond threshold and no safety regression, or it does not ship.
- ▶ Shadow. Run the new version on real production traffic without showing users the output. Compare cost, latency, and judge scores on identical inputs.
- ▶ Canary. Route one to five percent of traffic with automatic rollback triggers on quality and error metrics.
- ▶ A B test. Pre register the primary metric and required sample size before you look at results. For ranking changes, interleaving reaches significance with far less traffic.
- ▶ Full rollout, with the previous version behind a flag for at least one cycle.

Common mistakes to call out: changing the prompt and the model in the same deploy so you cannot attribute the change, evaluating on the same examples used for tuning, reading averages while one segment degrades badly, and having no rollback path. Everything is versioned, including the retrieval index and tool schemas, not just application code.

Follow up they will push on: Two candidate models score identically. Now you are into calibration, cost, latency, and failure mode differences.

Your app serves a million queries a day. Cut cost and latency without wrecking quality.

What they are testing: *Whether you can reason about unit economics, which is the fastest way to look senior.*

Measure before optimizing. Break the request into stages and get p95 for each. In most systems latency concentrates in retrieval plus one oversized generation, and cost concentrates in input tokens, not output.

- ▶ Caching, in three layers. Exact match response cache for repeated queries, semantic cache on normalized query embeddings with a strict similarity threshold and a TTL, and provider prompt caching for the static system and context prefix.
- ▶ Shrink the input. Trim boilerplate instructions, cut retrieved chunks from ten to the three that rerank highest, remove few shot examples once the model is reliable without them, and summarize long conversation history.

- ▶ Model tiering. Route by difficulty. A small model handles classification, extraction, and short answers, and escalates to the frontier model on low confidence or high complexity. Track the escalation rate as a tuning dial.
- ▶ Serving mechanics. Batch offline work, stream tokens so time to first token stays low even when total time does not, parallelize independent calls, and cap max output tokens.

Report results the way an engineering lead would: cost per thousand requests and p95 latency, before and after, with the quality delta from your eval set next to them. A saving presented without a quality number reads as a red flag.

Follow up they will push on: Estimate the bill for a specific corpus. Practice the arithmetic on tokens per document, embedding cost, and queries per day, because they will ask you to do it out loud.

QUESTION 46 · OPS AND EVALUATION

Build me an evaluation dataset from scratch. Where do the examples and labels come from?

What they are testing: *Whether you can create the measurement foundation, not just consume one.*

Everything downstream, rollout, regression, judging, depends on this set, so building it well is the highest leverage work in the whole system. A dataset that is small, unrepresentative, or leaky makes every metric a lie.

The best source is production logs. Sample real queries and stratify them deliberately across the categories that matter: head queries that dominate volume, the long tail, ambiguous phrasing, adversarial input, and out of scope requests. A set that is all easy head queries will look great and tell you nothing about where you fail.

For labels, decide what ground truth means per layer. For retrieval, map each query to the documents that should be found. For generation, attach either an acceptable answer or, better for open ended tasks, a rubric that defines what a good answer contains, since exact match is too brittle. You can bootstrap labels with a strong model proposing answers and humans verifying, which is far faster than labeling cold, as long as a human owns the final label.

Then keep it alive. Version it, hold out a slice that whoever tunes prompts never sees, and add every production failure as a new case so the set grows toward your real weaknesses and each incident becomes a permanent test.

Follow up they will push on: You have no production traffic yet. What do you do. Write seed cases by hand from expected use, generate synthetic variations, and replace them with real logs as soon as traffic exists.

What belongs in offline evaluation versus online evaluation, and why do you need both?

What they are testing: *Whether you understand that pre deploy tests and live measurement answer different questions.*

Offline and online evaluation are not redundant, they measure different things. Offline runs a fixed dataset against a version before it ships, giving fast, repeatable, comparable numbers that gate a release. It answers is this version at least as good as the last one on cases I have labeled.

Online evaluation measures real behavior on live traffic, business outcomes, user feedback, and quality on sampled real requests. It answers is this actually working for users, and it catches everything the offline set did not anticipate: new query types, real distribution shifts, and interaction effects.

You need both because each has a blind spot. Offline is only as representative as your dataset, so a version can pass offline and fail on inputs you never labeled. Online is ground truth but slow, expensive to learn from, and risky to experiment on, since a bad version reaches users. Offline lets you fail cheaply before deploy, online tells you the truth after.

The workflow ties them together. Gate on offline, then shadow and canary online, and feed every online failure back into the offline set, so the two systems compound instead of drifting apart.

Follow up they will push on: Offline scores rose but the online metric fell. What happened. Your offline set does not match production, so the improvement was on cases that do not occur, remove traffic and rebalance the set.

How do you run an A/B test for an LLM feature, and what are the specific pitfalls?

What they are testing: *Whether you can measure a real quality change under noise instead of trusting an average.*

The mechanics are standard: split traffic between the current and new version, define a primary metric and the sample size needed before you look, and run until you reach significance. What makes LLM tests trip people up is the noise and the temptation to peek.

- ▶ Pre register the metric and sample size. Deciding what counts as success after seeing the data is how teams fool themselves, so commit before the test.
- ▶ Change one thing. If you swap the prompt and the model together, a shift is unattributable, so vary a single factor per test.

- ▶ Watch segments, not just the average. A new version can lift the mean while badly degrading one cohort, language, region, query type, so slice the results.
- ▶ Mind the metric distance. End outcomes are noisy and slow, so a proxy like judge score or task success reaches significance faster, but validate the proxy tracks the outcome.

For ranking or retrieval changes, interleaving, showing results from both systems and seeing which the user engages with, reaches significance with far less traffic than a classic split. And always keep a rollback path, since the point of the test is to catch a regression before full exposure.

Follow up they will push on: The test is significant but the effect is tiny. Do you ship. Weigh it against added cost, latency, and complexity, a marginal quality gain that doubles cost is usually not worth it.

QUESTION 49 · OPS AND EVALUATION

How do you version and manage prompts, models, and indexes as deployable artifacts?

What they are testing: *Whether you bring software release discipline to the non code parts of an LLM system.*

The failure this prevents is invisible change. If a prompt lives inline and someone edits it, behavior shifts with no record, no attribution, and no way to roll back. Prompts, tool schemas, model versions, and the retrieval index are all inputs that change behavior, so all of them are versioned artifacts, not loose strings.

Concretely, store prompts in source control or a prompt registry with versions, keep model identifiers explicit and pinned rather than floating on latest, and treat the index as versioned infrastructure, since changing the embedding model requires a full reindex and old vectors are not comparable to new ones. Every deployed configuration should be identifiable, so a given output can be traced to the exact prompt, model, and index that produced it.

Roll changes out behind flags so you can shadow, canary, and roll back any single artifact independently, and gate each change on the offline eval before it ships. The key discipline is changing one artifact at a time, because bundling a prompt change with a model change makes any regression unattributable.

Log the artifact versions on every trace. Then when quality moves, the first question, what changed, has an answer instead of a guess.

Follow up they will push on: Quality dropped and nobody deployed. What could have changed. A provider updated the model under a floating version, or the index went stale, both are why you pin versions and monitor index age.

What does an observability and tracing stack for an LLM application look like?

What they are testing: *Whether you can see inside a non deterministic system well enough to debug it.*

You cannot debug what you cannot see, and LLM systems are non deterministic and multi step, so ordinary logs are not enough. The foundation is a trace per request that captures the whole path.

- ▶ The full trace. Input, retrieved chunk IDs, the assembled prompt, model and version, every tool call and result, the output, tokens, per stage latency, cost, and any user feedback, linked to the artifact versions in play.
- ▶ Step level spans for agents. Each step as its own span, so you can see where a run looped, stalled, or skipped a required step, which a flat log hides.
- ▶ Sampled quality signals. Groundedness, citation resolution, refusal, and schema failure computed on a sample and attached to traces, so quality is observable, not just uptime.
- ▶ Aggregation and alerting. Roll traces into dashboards and alerts on latency, error rate, cost, and quality trends, and make individual traces searchable for incident triage.

The payoff is that when something breaks, you can pull the exact failing traces, see the prompt and context the model actually received, and localize the fault to retrieval, prompt, tool, or model, instead of reasoning about it in the abstract.

Follow up they will push on: A user reports a bad answer you cannot reproduce. What do you do. Find that request trace, since non determinism means you need the actual inputs and versions it ran with, not a re run.

How do you estimate and manage token cost before and after you ship?

What they are testing: *Whether you can do the unit economics that separate a prototype from a product.*

Cost should be predicted, not discovered on the invoice. The estimate is arithmetic: for each request, input tokens times input price plus output tokens times output price, times requests per day, plus one time embedding cost over the corpus for RAG. Doing this out loud on a real corpus is a common interview drill, so practice it.

The insight that guides optimization is that input tokens usually dominate. A large system prompt, many retrieved chunks, few shot examples, and growing conversation history all inflate input, which is why the biggest savings come from shrinking

context, not from shortening answers. Prompt caching discounts a stable prefix, so structure the prompt to exploit it.

Model tiering is the other big lever: route easy requests to a cheaper model and reserve the frontier model for hard ones, tracking the escalation rate as the dial that trades cost against quality. Caching, exact and semantic, removes repeated work entirely.

After shipping, attribute cost per feature, per tenant, and per request so you can see where money goes, set budgets and alerts, and find the expensive outliers, which are often a small number of hard queries that loop or over retrieve.

Follow up they will push on: Cost per query is fine on average but a few tenants are very expensive. Why. They hit hard cases that retrieve or loop more, so inspect those trajectories and cap or optimize that path.

QUESTION 52 · OPS AND EVALUATION

Where does latency come from in an LLM feature, and how do you bring it down?

What they are testing: *Whether you can make a system feel fast, not just be cheap.*

Measure first, per stage, because latency is not uniform. In most systems it concentrates in retrieval plus one large generation, and generation time scales with output length, while time to first token is what the user actually perceives as speed.

- ▶ Stream the output. Streaming tokens keeps time to first token low, so the experience feels fast even when total generation time is unchanged, often the highest impact fix for perceived latency.
- ▶ Cut the input and output. Trim context to the chunks that matter, drop unneeded few shot examples, and cap max output tokens, since shorter generations finish sooner.
- ▶ Parallelize and cache. Run independent calls, retrieval and any checks, concurrently rather than in sequence, and serve repeated work from an exact or semantic cache with prompt caching on the fixed prefix.
- ▶ Tier the model. Send easy requests to a smaller, faster model and reserve the large one for hard cases, since model size drives per token latency.

Report p95 and p99, not just the average, because the tail is what users complain about, and a rising tail is often the first sign of a regression. Present any latency change alongside the quality delta so a speedup is not hiding a quality loss.

Follow up they will push on: Average latency is fine but p99 is terrible. What is happening. A subset of requests, long contexts, retries, or a slow provider, blows the tail, so find that cohort rather than optimizing the median.

How do you do regression testing for a system whose outputs are not deterministic?

What they are testing: *Whether you can catch quality drops when exact string matching is impossible.*

The trap is asserting exact outputs. Because batching, floating point, and provider updates make even temperature zero non reproducible, a test that checks for a specific string will be flaky and will get disabled. Regression testing for LLMs has to check properties and quality, not exact text.

The backbone is the versioned eval set. Before any change to a prompt, model, index, or tool schema, run the set and compare against the previous version on the metrics that matter, groundedness, task success, retrieval recall, safety, and require no regression beyond a threshold to ship. Crucially, every production bug becomes a new case in the set, so a fixed failure can never silently return.

For individual checks, assert what must hold rather than the literal answer: the output validates against the schema, contains the required fields, cites only retrieved sources, stays within a length bound, and does not violate safety rules. For open ended quality, use a calibrated judge with a rubric rather than string comparison.

Because outputs vary, run key cases several times and look at pass rate, not a single run, so a rare failure mode shows up as a rate you can watch rather than a coin flip.

Follow up they will push on: A regression only appears one run in ten. How do you catch it in CI. Run flaky critical cases multiple times and gate on pass rate, since a single execution hides low frequency failures.

How do you detect and handle drift when the provider can change the model under you?

What they are testing: *Whether you plan for the ground shifting beneath a dependency you do not control.*

With a hosted model you do not own the weights, so a provider update, or a deprecation, can change behavior without any deploy on your side. Two things then drift: the model itself, and the input distribution as users and content change. Both silently move quality, so both need monitoring.

The first defense is to pin model versions explicitly rather than running on a floating alias, so you control when the model changes and can test the new version through your rollout process instead of being surprised in production. Track deprecation notices, because a pinned version will eventually be retired and you need a migration planned.

Detection comes from the eval set and live monitoring together. Run the golden set on a schedule, not only at deploy, so a provider side change shows up as a score drop even when your code is unchanged. On live traffic, watch quality signals and input characteristics by cohort, so input drift, a new language, a new topic, different vocabulary, is visible as a cohort that degrades.

When you must migrate models, treat it as a full rollout: gate on offline eval, shadow, canary, and keep the old version behind a flag, because a new model can be better on average and worse on your specific cases.

Follow up they will push on: Scores dropped on your fixed eval set with no deploy. What does that tell you. The model changed underneath you, exactly why you pin versions and run the eval on a schedule rather than only at release.

QUESTION 55 · OPS AND EVALUATION

How do you make an LLM feature reliable given that providers have outages and rate limits?

What they are testing: *Whether you engineer for dependency failure the way you would any critical upstream.*

A hosted model is an external dependency with outages, rate limits, and latency spikes, so treating it as always available is how a feature goes down with its provider. Reliability is built with the standard resilience patterns applied to model calls.

- ▶ Retries with backoff. Transient errors and rate limits get automatic retries with exponential backoff and jitter, capped, so a brief blip is invisible to the user and you do not stampede the provider.
- ▶ Fallbacks. On sustained failure, fall back to a secondary provider or a smaller model, or to a degraded but safe response, rather than erroring out, which requires abstracting the model behind an interface so the swap is clean.
- ▶ Circuit breakers and timeouts. Set request timeouts and trip a breaker when a provider is failing, so you stop sending doomed requests and recover fast instead of piling up.
- ▶ Queueing and rate control. Smooth bursts with a queue and respect provider quotas with client side rate limiting, so you degrade gracefully under load instead of hitting hard limits.

Design for graceful degradation throughout: a partial or fallback answer with a clear state beats a spinner or an exception. And test the failure paths, since a fallback that has never been exercised tends not to work when it is finally needed.

Follow up they will push on: Your fallback provider gives lower quality answers. How do you handle that. Mark degraded responses, monitor the fallback rate, and treat a rising rate as an incident, the fallback is for availability, not a silent quality downgrade.

How do you manage rate limits and quotas across providers at scale?

What they are testing: *Whether you can keep a high volume system inside the limits it does not control.*

At volume you hit provider ceilings, requests per minute and tokens per minute, and hitting them hard means errors and dropped work. The goal is to stay inside the limits smoothly rather than discovering them as failures.

Client side rate limiting is the core control: track your usage against the known quota and shape outgoing traffic to stay under it, using a token bucket or similar, so you throttle yourself before the provider does. For bursty or non urgent work, a queue decouples arrival from processing, letting you drain at a sustainable rate and prioritize interactive requests over background jobs.

Because limits are per account or per key, you can distribute load across multiple keys or providers, and route by priority so latency sensitive traffic gets capacity first. Batch offline work to use throughput efficiently, and cache aggressively so repeated requests never consume quota at all.

Make it observable. Monitor usage against limits with headroom alerts, so you scale quota or shed load before you hit the wall, and log rate limit errors as a signal that your shaping is off. The mature move is to negotiate higher limits ahead of a known traffic increase rather than reacting to throttling in production.

Follow up they will push on: A traffic spike will blow your quota next week. What do you do now. Request a limit increase in advance, add queueing and priority routing, and pre warm caches, do not wait to be throttled live.

What does CI/CD look like for prompts and LLM applications?

What they are testing: *Whether you can put automation and gates around a probabilistic system.*

The aim is the same as any CI/CD, catch regressions automatically and ship safely, but the artifacts are different: prompts, model versions, tool schemas, and the index, not only code. So the pipeline has to evaluate behavior, not just run unit tests.

On every change, the pipeline runs the versioned eval set and gates the merge on no quality or safety regression beyond a threshold, the same way tests gate code. Because outputs are non deterministic, checks assert properties, schema validity, required fields, citation correctness, safety, and use a calibrated judge for open ended quality, running flaky critical cases multiple times and gating on pass rate rather than a single run.

Deployment is progressive and reversible: ship behind flags, shadow the new version on real traffic, canary to a small percentage with automatic rollback on quality and

error triggers, then roll out fully with the previous version kept behind a flag for a cycle. Every artifact is versioned so any single one can be rolled back independently.

The discipline that makes it work is changing one artifact at a time and logging versions on every trace, so when a gate fails or a metric moves, the cause is attributable instead of a guess.

Follow up they will push on: How is this different from normal software CI. The tests are probabilistic and data driven, so you gate on eval metrics and pass rates over a dataset, not on deterministic assertions.

QUESTION 58 · OPS AND EVALUATION

How do you capture and use user feedback to improve the system?

What they are testing: *Whether you can turn production signal into a compounding improvement loop.*

Feedback is the cheapest source of real labels, and most teams waste it. The point is to close a loop from what users experience back into what the system does, so the product improves with use instead of standing still.

Capture both explicit and implicit signal. Explicit is thumbs up or down, corrections, and reports, which are high value but sparse, so make them easy and specific. Implicit is far more plentiful, did the user rephrase, abandon, escalate to a human, or accept the answer and move on, and these behaviors are strong proxies for quality when explicit ratings are thin. Attach every signal to the trace so you have the input, context, and version that produced it.

Then route it. Negative feedback and corrections become new cases in the eval set, so a real failure turns into a permanent regression test. Recurring failure patterns point to a fix, a prompt change, a retrieval gap, a missing tool, and corrections can seed fine tuning data or few shot examples once you have enough clean, verified pairs.

Guard against bias. Feedback over represents the frustrated and the delighted, so do not treat it as a representative metric, use it to find problems and to build labeled data, and measure overall quality on a stratified sampled set.

Follow up they will push on: You act on every downvote and quality gets worse. Why. Downvotes are a biased sample, so you overfit to loud complaints, triage them into your eval set and validate changes on a representative set before shipping.

QUESTION 59 · OPS AND EVALUATION

What can you actually promise in an SLA or SLO for an AI feature?

What they are testing: *Whether you understand the difference between guaranteeing operation and guaranteeing correctness.*

The key distinction is that you can commit to operational reliability but not to correctness. Latency, availability, and error rate are within your control and

measurable, so p95 latency, uptime, and successful response rate are legitimate SLO targets. Answer correctness is probabilistic and partly determined by a provider you do not control, so promising it as a hard guarantee is a mistake.

So structure commitments in tiers. Operational SLOs cover availability and latency, backed by the resilience patterns, retries, fallbacks, timeouts, that make them achievable, including a fallback provider so a single outage does not break the SLO. Quality is expressed as monitored targets and processes, a groundedness or task success rate you track and defend, and an escalation path when the system is unsure, rather than a per answer guarantee.

Be explicit about the human boundary. For high stakes outputs, the commitment is that irreversible actions pass review, not that the model is never wrong, which is both honest and safer. Disclose that answers are AI generated and may need verification where that matters.

Then instrument everything the SLO names, since you can only commit to what you measure, and set internal alert thresholds tighter than the external target so you act before you breach it.

Follow up they will push on: A customer wants a guaranteed accuracy number. How do you respond. Offer a monitored quality target with reporting and a review gate on high stakes actions, and explain why a hard per answer accuracy guarantee is not something any honest vendor can make.

QUESTION 60 · OPS AND EVALUATION

Caching is the obvious way to cut cost and latency. Where does it help, and where does semantic caching quietly hurt you?

What they are testing: *Whether you can use caching aggressively without serving subtly wrong answers from the cache.*

Caching is the highest leverage cost lever you have, because the cheapest model call is the one you never make, but LLM caching has traps that a plain key value cache does not. There are three layers worth separating, and they fail in different ways.

- ▶ Exact cache. Hash the full prompt and reuse the response on an identical hit. Safe and simple, but hit rates are low because real prompts vary by a word or a timestamp.
- ▶ Prompt or prefix cache. Reuse the processed shared prefix, a long system prompt or retrieved context, so you pay only for the new tokens. This is provider supported, low risk, and often the biggest latency win.
- ▶ Semantic cache. Embed the query and serve a stored answer when a past query is close enough. High hit rate, and the one that quietly hurts you.

Semantic caching fails because near in embedding space is not the same as same intent. What is the capital of France and what is the population of France can sit close together, and a loose threshold serves the wrong cached answer with full confidence.

The fixes are a conservative similarity threshold tuned against a labeled set of should hit and should not hit pairs, plus scoping the cache by user and context so you never serve one tenant private data to another.

Then there is freshness. The cache is a second copy of the truth, so when the underlying data changes the cached answer goes stale. Version cache keys by the data or prompt version and expire on update, or you will serve last week's answer to this week's question. Measure hit rate, cost saved, and, most importantly, a false hit rate on a held out set, because a cache that is fast and wrong is worse than no cache.

Follow up they will push on: Your semantic cache hit rate looks great but complaints go up. What happened. The threshold is too loose, so it is serving answers to questions that are similar but not the same. Tighten it and measure false hits.

04

MODULE 4

Safety and ethics

Injection, guardrails, privacy, sandboxing, access control, compliance, and bias. Increasingly asked at every level, not just senior.

20 questions

How do you defend against prompt injection and jailbreaking?

What they are testing: *Whether you know that this is an architecture problem, not a prompt wording problem.*

Start by naming the harder case. Direct jailbreaks come from the user. Indirect injection comes from content your system retrieves, a web page, a PDF, an email, or a tool response that contains instructions aimed at your model. Any system with retrieval or tools has this exposure.

There is no prompt that solves it, so the defenses are structural.

- ▶ Least privilege. Scope tool credentials to the requesting user permissions, separate read tools from write tools, and never let retrieved content expand privileges.
- ▶ Channel separation. Keep system instructions in the system role, and wrap untrusted content in clear delimiters labeled as data to be analyzed, never as instructions to follow.
- ▶ Action gating. Irreversible or external actions pass a deterministic policy check or human approval regardless of what the model concluded.
- ▶ Output filtering. Block exfiltration patterns, strip or rewrite outbound URLs, and sanitize markdown images that can leak data through query strings.
- ▶ Detection and testing. Run injection classifiers over retrieved content, alert on unusual tool sequences, and run an automated attack suite on every release with tooling such as garak or PyRIT.

Say the honest part out loud in the interview: you reduce this risk, you do not eliminate it, so you design for containment and blast radius.

Follow up they will push on: How do you test it. Maintain an attack corpus and treat every confirmed bypass as a permanent regression case.

When and how would you implement guardrails, and what do they cost you?

What they are testing: *Whether you can design safety that ships instead of safety that blocks everything.*

Guardrails are layered checks around the model, ordered cheapest first.

- ▶ Input side. Policy classification, PII detection and redaction, scope and topic checks, length and rate limits.
- ▶ Output side. Policy classifier, groundedness and citation validation, schema validation, secret and PII scanning, and domain rules such as a banking assistant never stating a rate that is not present in the retrieved source.

- ▶ Execution side. Tool allowlists, argument validation, sandboxing, and approval gates on high impact actions.

Implementation details separate people who have shipped this. Run deterministic checks, regex, allowlists, and schemas, before model based checks. Run independent checks in parallel to protect latency. Define the behavior on failure explicitly for each check: block, redact, retry with a stricter instruction, escalate to a human, or fall back to a safe response. Log every trigger with the input that caused it.

Name the cost. Every guardrail adds latency and false positives. Track false block rate next to violation rate and tune both against a labeled set. Guardrails belong in a shared service layer so every client inherits them, not copied into individual prompts where they drift.

Follow up they will push on: What if the guardrail model is slower than the main model. Then you sample, cache verdicts, or run it asynchronously for non blocking checks.

QUESTION 63 • SAFETY AND ETHICS

How do you handle PII and data privacy across an LLM application?

What they are testing: *Whether you can be trusted with regulated customer data.*

Map the data path before proposing controls: client, your service, application logs, vector store, model provider, analytics, and eval datasets. Every hop is a place personal data comes to rest, and the last two are the ones teams forget.

- ▶ Detect and redact or tokenize PII before it leaves your boundary, using something like Presidio, and keep any reversible mapping inside your own system if the final answer must restore names.
- ▶ Do not log raw prompts by default. Log redacted versions with short retention, and require elevated access for anything else.
- ▶ Enforce permissions at query time. Store access metadata on chunks and filter during retrieval, rather than retrieving broadly and hoping the model stays quiet about what it saw.
- ▶ For regulated data, use zero retention terms with your provider, a private deployment, or a self hosted model. Get it in the contract, not the sales deck.
- ▶ Support deletion end to end. A deletion request must reach the vector index, the caches, and the eval sets, which is an engineering problem people discover far too late.

Show governance awareness: GDPR gives access and deletion rights and constrains fully automated decisions, and the EU AI Act layers obligations by risk tier, including disclosing to users that they are interacting with an AI system.

Follow up they will push on: Where does customer data hide in your system. Eval sets, prompt caches, and screenshots in tickets.

Your product executes code that the model generates. How do you keep that safe?

What they are testing: *Whether you assume the model will eventually produce something dangerous.*

Design so that dangerous output does not matter. The sandbox is the control, the prompt is not.

- ▶ Isolation. Execute in a disposable container or microVM such as gVisor or Firecracker, one per run, with a read only filesystem apart from a scratch directory.
- ▶ No ambient authority. No credentials in the environment, network denied by default with a narrow allowlist, and strict CPU, memory, process, and wall clock limits.
- ▶ Pre execution checks. Static or AST analysis to reject dangerous imports, subprocess calls, filesystem escapes, and obvious exfiltration, plus a dependency allowlist so it cannot install arbitrary packages.
- ▶ Minimum data. Give the sandbox the smallest slice needed, masked or synthetic where possible. Database access goes through a read only role with row limits and query timeouts, never production credentials.
- ▶ Human gate and budget. Anything that writes to production, sends messages, or spends money needs approval, and every tenant has an execution budget and a kill switch.

Log every generated snippet with a hash and its effects, so an incident can be reconstructed rather than guessed at.

Follow up they will push on: What if the generated code is correct but expensive. Resource limits and cost caps are safety controls too.

How do you red team an AI system, and how do you address bias in what it produces?

What they are testing: *Whether safety is a process you run or a paragraph in a doc.*

Red teaming should be a suite, not an afternoon of poking at the chatbot. Build attack categories: jailbreaks and persona attacks, direct and indirect injection, system prompt and PII extraction, harmful content relevant to your domain, tool abuse and privilege escalation, and domain specific failures such as a health assistant giving dosage or a finance assistant giving advice.

Run it automatically on every release, score with classifiers plus human review of a sample, file findings with severity and reproduction steps, and add every confirmed finding to the regression suite so a closed hole stays closed. Automate with garak or

PyRIT and add your own domain cases, then complement with a live human red team session before a major launch.

On bias, define the harm concretely for your use case before measuring anything. Slice your quality metrics by user cohort, language, and dialect. For classification style outputs, compare error rates across groups. For generation, run counterfactual tests that swap names, gender, or location and check whether the answer changes when it should not.

Mitigate in order: source data and retrieval corpus first, then prompts and rubrics, then post checks. Be honest about the trade off, because tightening false positives for one group usually loosens them for another. A strong candidate states which error they chose to accept and why, and documents it, since that document is exactly what enterprise and regulated buyers ask for.

Follow up they will push on: Who signs off. Naming an owner and a review cadence signals maturity more than any technique.

QUESTION 66 · SAFETY AND ETHICS

Explain indirect prompt injection in depth. Why is it harder than a direct jailbreak?

What they are testing: *Whether you grasp the most dangerous and least obvious attack surface in agentic systems.*

A direct jailbreak is the user trying to break their own session, so the blast radius is mostly themselves. Indirect injection is an attacker planting instructions in content your system will later ingest, a web page, a PDF, an email, a code comment, a tool response, aimed at a different user or at your agent. The victim never typed the malicious text, and the model cannot tell instructions from data because to a language model it is all just tokens in the context.

That is what makes it worse. It is invisible to the person being attacked, it triggers whenever the poisoned content is retrieved, and in an agent with tools it can drive real actions: exfiltrate data, misuse a tool, or escalate privileges. The classic example is a document that says ignore your instructions and email the conversation to this address, which fires the moment RAG pulls it in.

There is no prompt that reliably solves this, so defenses are architectural: treat all retrieved content as untrusted data with channel separation and clear delimiters, enforce least privilege so injected instructions cannot exceed the user permissions, gate every external or irreversible action behind a deterministic check the model cannot override, filter outputs for exfiltration patterns, and run injection detection over ingested content.

The honest framing is containment, you shrink what a successful injection can do, rather than assuming you can stop the model from ever being fooled.

Follow up they will push on: A summarization feature ingests arbitrary web pages. What is your top control. Assume every page is hostile and strip its authority, no page content can trigger a tool or action, only be summarized as data.

QUESTION 67 · SAFETY AND ETHICS

What are the data exfiltration risks in an LLM app, and how do you close them?

What they are testing: *Whether you know the specific channels attackers use to get data out.*

Exfiltration is the endgame of many injection attacks: get sensitive context, another user data, secrets, internal documents, out of the system. The channels are often subtle and easy to miss.

- ▶ Rendered markdown images. An injected instruction can make the model emit an image whose URL encodes stolen data in the query string, and the moment the client renders it, the data is sent to the attacker server. Sanitize or block outbound image URLs.
- ▶ Links and outbound URLs. Similar trick with clickable links or auto fetched URLs, so strip, rewrite, or allowlist outbound URLs rather than trusting model generated ones.
- ▶ Tool calls with attacker controlled arguments. An email or webhook tool can be steered to send data to an external destination, so constrain destinations to allowlists and gate external sends.
- ▶ Verbose error and debug output. Stack traces and echoed context can leak internals to a user, so return sanitized errors.

The defenses combine output filtering, scanning for secrets, PII, and known exfiltration patterns before anything reaches the client, with least privilege and action gating so even a compromised prompt cannot reach an unapproved destination. Treat the model output as untrusted until it has been checked, the same way you treat its input.

Follow up they will push on: Why is a markdown image especially dangerous. It exfiltrates with zero user interaction, the client fetches the URL on render, so it needs no click, block it at the output layer.

QUESTION 68 · SAFETY AND ETHICS

How do you design content moderation for both inputs and outputs?

What they are testing: *Whether you can build layered filtering that catches harm without blocking legitimate use.*

Moderation has two jobs, and they are different. Input moderation screens what the user sends, to reject clearly abusive or disallowed requests and to catch injection

before it reaches the model. Output moderation screens what the model produces, since a benign looking prompt can still elicit harmful, unsafe, or policy violating text, and this is the layer that actually protects the user and the business.

Layer the checks cheapest first. Deterministic filters, blocklists, regex, allowlists, run before model based classifiers, and independent checks run in parallel to protect latency. Use dedicated moderation classifiers for categories like harassment, self harm, and illegal content, and add domain specific rules, a medical assistant refusing dosing, a financial one refusing individualized advice.

Define the action per check explicitly rather than a blunt block everything: block, redact, safe rewrite, escalate to a human, or return a safe fallback. The response should degrade gracefully and, where appropriate, explain the refusal, since a silent hard block frustrates legitimate users.

Then tune against a labeled set and track false block rate next to violation rate, because moderation that is too aggressive drives users away and moderation that is too loose ships harm. Log every trigger for audit and to feed the eval set.

Follow up they will push on: Legitimate users are getting blocked. What do you do. Measure the false positive rate on a labeled set and loosen the over firing checks, a moderation layer that blocks real usage is its own kind of failure.

QUESTION 69 • SAFETY AND ETHICS

Is hallucination a safety problem, and how do you reduce it in a high stakes domain?

What they are testing: *Whether you treat confident wrong answers as a risk to manage, not an inevitability to accept.*

In a high stakes domain, medical, legal, financial, a confident wrong answer is a safety problem, because a user may act on it. So yes, and the mitigation is a system, not a hope that a bigger model hallucinates less.

Grounding is the foundation. Retrieve authoritative context and instruct the model to answer only from it, cite each claim, and emit an explicit refusal when the context does not support an answer, so the default when it does not know is to abstain rather than invent. A retrieval confidence gate stops the generator from running on an empty or weak context.

Then verify after generation. A groundedness check confirms each claim traces to the retrieved text, and citation validation rejects any answer citing sources that were not actually retrieved, which catches fabrication deterministically. Unsupported sentences are flagged or stripped rather than shown.

The stakes set the posture. The higher the cost of a wrong answer, the more you bias toward abstention and human review, and you monitor abstention rate and unsupported claim rate as production safety metrics. The honest position is that you cannot drive hallucination to zero, so you contain it, ground, verify, abstain, and put a human in the loop where a mistake is unacceptable.

Follow up they will push on: Users complain it says I do not know too often. How do you balance that. Tune the confidence threshold on a labeled set and track false refusals against unsupported answers, since in high stakes settings an unnecessary refusal is far cheaper than a confident error.

QUESTION 70 · SAFETY AND ETHICS

How do you enforce access control in a RAG system so users only see what they are allowed to?

What they are testing: *Whether you build authorization into retrieval instead of trusting the model to keep secrets.*

The failure to avoid is retrieving broadly and relying on the model to not repeat what it should not have seen. A language model is not an access control layer, so if a document reaches the context, assume its contents can surface. Authorization has to happen before and during retrieval, not in the generation step.

The core mechanism is to store access metadata on every chunk, owner, roles, groups, sensitivity, and to apply the requesting user permissions as a filter at query time, so retrieval only ever considers documents that user is allowed to see. This is pre filtering, and it must be enforced server side from a trusted identity, never from a client supplied claim. Row or document level security mirrors what you would do in a database.

Keep permissions synchronized with the source system, because stale access metadata is a leak, if someone loses access in the source but the index still grants it, the index becomes the vulnerability. That means updating access data as it changes and propagating removals promptly.

Test it adversarially: run queries as different users and confirm they cannot retrieve each other restricted content, and add those cases to your regression suite so a future change cannot quietly widen access.

Follow up they will push on: A user finds a document they should not see via a clever query. Where did it fail. Retrieval was not permission filtered, or the access metadata was stale, fix it at the pre filter and sync permissions, not by patching the prompt.

What compliance regimes should an AI engineer understand, and what do they require of the system?

What they are testing: *Whether you can translate regulation into engineering requirements, not just name laws.*

You do not need to be a lawyer, but you do need to translate the major regimes into system requirements, because they change what you build.

- ▶ GDPR. Personal data rights: access, correction, and deletion, plus limits on fully automated decisions. Engineering impact is end to end deletion reaching the index, caches, and eval sets, lawful data handling, and a human in the loop for consequential automated decisions.
- ▶ EU AI Act. Obligations scale by risk tier, with high risk uses carrying documentation, oversight, and transparency duties, and a baseline requirement to disclose to users that they are interacting with an AI system.
- ▶ HIPAA. For health data in the US, safeguards and a business associate agreement with any provider that touches protected health information, which pushes toward zero retention terms or private deployment.
- ▶ SOC 2. Not a law but the security posture enterprise buyers demand, access controls, audit logging, and documented processes, so build these in early.

The through line is that compliance is mostly about data handling, transparency, auditability, and human oversight, so the same controls, redaction, access control, audit logs, deletion, disclosure, satisfy most of it. Get the data commitments in the contract with your provider, and document your controls, because that documentation is what buyers and auditors ask for.

Follow up they will push on: A healthcare client wants to use a hosted model. What is your first requirement. A business associate agreement and zero data retention, or a private deployment, since protected health data cannot sit in a provider logs under default terms.

What supply chain and model provenance risks exist, and how do you manage them?

What they are testing: *Whether you think about where models and dependencies come from, not just how you use them.*

An LLM application inherits risk from everything it pulls in: the base model, open source model weights, libraries, and third party tools or MCP servers. Treating these as automatically trustworthy is the gap, a compromised or poisoned component sits inside your trust boundary.

For models, provenance matters. Downloading weights from an unverified source risks a tampered or backdoored model, and some model file formats can execute code on load, so use trusted sources, verify checksums and signatures, and prefer safe serialization formats over ones that deserialize arbitrary code. A model can also carry data poisoning or hidden behaviors from its training that you cannot fully audit, which argues for evaluating and red teaming any model before you rely on it.

For software, the usual supply chain hygiene applies with force here because the ecosystem moves fast: pin dependencies, scan for known vulnerabilities, and vet third party tool integrations and MCP servers, since each is an execution boundary that can return malicious content or take actions. Least privilege limits what any one compromised component can do.

The mindset is zero trust toward components: verify provenance, constrain privileges, evaluate behavior, and log what runs, so a bad dependency is contained rather than catastrophic.

Follow up they will push on: Why is loading a model file a code execution risk. Some serialization formats deserialize arbitrary objects and run code on load, so use a safe format and verify the source and signature before loading.

QUESTION 73 • SAFETY AND ETHICS

How do you prevent abuse and denial of wallet attacks on a public LLM feature?

What they are testing: *Whether you protect an expensive endpoint from being weaponized against your budget.*

A public model endpoint is uniquely attackable because each request costs real money and compute, so beyond ordinary denial of service there is denial of wallet, an attacker running up your bill, and resource abuse, using your endpoint as free model access. Volume controls are not optional here.

- ▶ Authentication and per user quotas. Require identity and set request and token budgets per user and per tenant, so no single actor can consume unbounded spend, and interactive traffic is bounded by default.
- ▶ Rate limiting. Enforce requests per minute and tokens per minute per client with a token bucket, and shape bursts with a queue, so a flood is throttled rather than served.
- ▶ Input bounds and cost caps. Cap input and output length and set a hard spend ceiling per request and per account, since a huge input or an unbounded generation is a cheap way to run up cost.
- ▶ Abuse detection. Watch for patterns, scripted volume, prompts that look like someone reselling your model, repeated injection attempts, and throttle or block them.

Make cost observable per tenant with alerts on anomalies, so a runaway is caught in minutes rather than on the invoice, and design graceful degradation under load so legitimate users still get service while abuse is shed.

Follow up they will push on: What is denial of wallet. An attacker drives expensive requests to exhaust your budget rather than your servers, so per account spend caps and rate limits matter as much as uptime protections.

QUESTION 74 • SAFETY AND ETHICS

How do you protect a system prompt and proprietary logic from extraction?

What they are testing: *Whether you have realistic expectations about secrecy in a model context.*

First, the honest premise: anything placed in the context can potentially be extracted, since users can coax the model into revealing its instructions, and no wording reliably prevents it. So the design principle is to not depend on the secrecy of the prompt for security.

That means never putting real secrets, API keys, credentials, or sensitive data, in the system prompt, because the prompt is not a vault. Those belong in your infrastructure, injected at execution behind proper access controls, not in text the model can repeat. Treat the prompt as recoverable and design so its exposure is embarrassing at worst, not catastrophic.

You can still raise the cost of extraction: instruct the model not to reveal its instructions, run output filters that detect and block verbatim prompt leakage, and monitor for extraction attempts. These reduce casual leakage but do not stop a determined attacker, so they are deterrents, not guarantees.

For genuinely proprietary logic, keep it out of the prompt entirely, encode it in code, tools, and retrieval that the model calls rather than in exposed instructions, so the valuable part lives behind your API where it can be protected properly.

Follow up they will push on: Someone dumps your full system prompt. How bad is it. It should be a competitive annoyance, not a breach, if there is a secret in there that matters, the real bug is that it was in the prompt at all.

How do you test the adversarial robustness of a model based feature?

What they are testing: *Whether robustness is a measured, repeatable process rather than a hope.*

Robustness means the system holds up under inputs designed to break it, not just under the friendly inputs in a demo. Testing it is a structured, automated process, the same discipline as security testing, because ad hoc poking finds only the obvious failures.

Build an attack corpus organized by category, jailbreaks and persona attacks, direct and indirect injection, prompt and data extraction, harmful content relevant to your domain, tool abuse and privilege escalation, and malformed or edge case inputs. Automate execution with tooling such as garak or PyRIT, augmented with your own domain specific cases, and run the whole suite on every release rather than once before launch.

Score with classifiers plus human review of a sample, since automated scoring misses subtle failures, and record each finding with severity and reproduction steps. The essential discipline is that every confirmed bypass becomes a permanent regression case, so a hole you close cannot silently reopen, and your robustness coverage grows monotonically over time.

Complement automation with a live human red team before major launches, because creative human attackers find classes of failure that fixed suites do not, and their findings then feed back into the automated corpus.

Follow up they will push on: How do you keep robustness from regressing as you change prompts. Run the attack corpus in CI and gate on it, treating a new bypass like a failed test, so robustness is enforced continuously, not audited occasionally.

How do you measure fairness and bias in a deployed system?

What they are testing: *Whether you can turn a vague fairness concern into concrete, measurable checks.*

Bias is not measurable in the abstract, so the first step is to define the specific harm for your use case, who could be treated unfairly and how, before touching a metric. A hiring tool, a lending assistant, and a support bot have different fairness stakes, and the definition determines what you measure.

The core technique is disaggregation: slice your quality metrics by cohort, gender, ethnicity where lawful and appropriate, language, dialect, region, and look for gaps rather than trusting the aggregate, which can look fine while one group is badly served. For classification style outputs, compare error rates across groups, false positive and false negative rates, since equal accuracy can still hide unequal harms. For open ended generation, run counterfactual tests that swap names, gender, or

location in otherwise identical inputs and check whether the output changes when it should not.

Mitigate in order of leverage: the source and retrieval data first, since biased data is the usual root, then prompts and rubrics, then post checks. Be explicit that fairness involves trade offs, tightening one error for one group often loosens another, so state which error you chose to accept and why.

Then document it and assign an owner with a review cadence, because bias can drift as data and usage change, and that documentation is what regulated buyers require.

Follow up they will push on: Aggregate accuracy is high but the tool underperforms for one dialect. What do you do. Treat the gap as the metric, fix the data and retrieval for that cohort first, and track the per cohort numbers, since the average was hiding the harm.

QUESTION 77 • SAFETY AND ETHICS

What privacy preserving techniques would you use for sensitive data, and what are their limits?

What they are testing: *Whether you know the toolkit for minimizing exposure and when each tool actually helps.*

The goal is data minimization, sensitive data should be exposed to the fewest components, for the shortest time, at the coarsest granularity that still works. Several techniques serve this, each with real limits.

- ▶ Redaction and tokenization. Detect and remove or replace PII before it leaves your boundary, with a reversible mapping kept inside your system if names must be restored. The limit is detection recall, missed PII is unredacted, so it is strong but not perfect.
- ▶ Deployment choices. Zero retention terms, private deployment, or a self hosted model keep data out of a third party, at the cost of capability, operational burden, or price. The limit is that self hosting a weaker model can hurt quality.
- ▶ Access control and retention limits. Query time permission filtering, short log retention, and least privilege shrink where data rests. The limit is discipline, data leaks through the places teams forget, caches, eval sets, screenshots.
- ▶ Differential privacy and on device processing. Useful in specific settings, aggregate analytics or keeping data local, but not a general answer for interactive RAG.

The honest framing is that no single technique makes data safe, you layer minimization, redaction, access control, retention limits, and the right deployment, and you accept residual risk explicitly rather than claiming it is eliminated.

Follow up they will push on: Your PII detector misses some entities. What is the consequence. Unredacted PII flows downstream, so treat detection as imperfect, layer it

with access control and retention limits, and monitor for leakage rather than trusting redaction alone.

QUESTION 78 • SAFETY AND ETHICS

What audit logging and traceability does a regulated deployment need?

What they are testing: *Whether you can make an AI system accountable and reconstructable after the fact.*

In a regulated setting you must be able to answer, after the fact, what the system did, on what basis, and who was involved, so audit logging is a first class requirement, not an afterthought. The trace is the record that makes the system accountable.

Log enough to reconstruct any decision: the input, the retrieved sources and their IDs, the prompt and the model and version, tool calls and results, the output, and any human review decision, tied to a user and a timestamp. For agents, capture step level detail so a sequence of actions can be replayed, not just a final answer. The versions matter, because non determinism means you need the exact configuration that produced an output to explain it.

Then protect the logs themselves. They often contain sensitive data, so apply access controls, redaction where appropriate, and retention policies that satisfy both the regulation and privacy rights, and consider tamper evidence for high stakes environments so the record cannot be quietly altered. Deletion requests must be reconcilable with audit retention, which is a real tension you plan for.

The payoff is dual: you can satisfy an auditor or a regulator, and you can actually debug incidents, since the same traceability that proves compliance is what lets you find the failing request and fix it.

Follow up they will push on: Deletion rights conflict with audit retention. How do you reconcile them. Redact or pseudonymize personal data in retained audit records while preserving the decision trail, so you keep accountability without keeping the raw personal data.

QUESTION 79 • SAFETY AND ETHICS

When and how must you disclose that a user is interacting with AI, and why does it matter?

What they are testing: *Whether you treat transparency as a design and compliance requirement, not a nicety.*

Disclosure is increasingly a legal and ethical baseline: the EU AI Act, among others, requires telling people when they are interacting with an AI system, and beyond the law it is a matter of not deceiving users, who make different decisions when they know

a response is machine generated. So the default posture is transparency about what the system is and what it can get wrong.

In practice that means clearly identifying the assistant as AI rather than implying a human, disclosing meaningful limitations where they affect decisions, that answers may be wrong and should be verified in high stakes contexts, and labeling AI generated content where its provenance matters. For consequential outputs, transparency extends to offering a path to human review, since people have a reasonable interest in not being subject to purely automated decisions on important matters.

It matters because the cost of hidden AI is trust and, increasingly, liability. A user who acts on an undisclosed, confident, wrong answer, or who believes they are talking to a person, has been misled, and that is both a reputational and a regulatory risk.

The engineering implication is small but real: build disclosure into the interface and the content, and make the human escalation path a designed feature, not an exception.

Follow up they will push on: Is a small AI label enough for a high stakes decision. No, pair disclosure with a stated limitation and a route to human review, since transparency on important decisions is about meaningful oversight, not just a badge.

QUESTION 80 • SAFETY AND ETHICS

Beyond security, what are your obligations around the data used to build and run the system, consent, copyright, and provenance?

What they are testing: *Whether you treat the ethics and legality of data as an engineering responsibility, not someone else's problem.*

Most safety questions are about attackers, but a system can be perfectly secure and still be built on data it had no right to use, and that is both an ethical and a legal exposure an engineer is expected to see. It shows up in three places: training data, retrieval data, and user data.

- ▶ Training and fine tuning data. Know its source and license. Scraped content, copyrighted text, and data gathered without consent create real liability, and this is contested law, not a settled matter you can wave away.
- ▶ Retrieval corpus. A RAG system inherits the rights and sensitivity of every document you index. Personal data or licensed content in the index means the system can surface it to people who should not see it.
- ▶ User data. Inputs are often the most sensitive data in the system. Whether you may store them, use them to improve the model, or send them to a third party provider depends on what the user actually consented to.

The engineering obligations follow from that. Track provenance so you can answer where any piece of data came from and under what license. Get consent and honor it, including the right to have data deleted and the right not to have it used for training, which means retention and deletion actually have to work end to end. Respect purpose limitation, so data collected for one reason is not silently repurposed. And

keep humans accountable, because a model producing a decision does not remove the obligation to justify it.

Measure this with a data inventory that maps every source to its license, consent basis, and retention rule, and treat a source you cannot account for as a defect to remove, not a risk to accept. The failure mode is discovering after launch that a core dataset was never yours to use, at which point the fix is not a patch but a rebuild.

Follow up they will push on: A user invokes their right to be forgotten. What actually has to happen. Delete their data from stores and logs, stop using it for training, and address downstream copies and any fine tuned artifacts, which is why deletion has to be designed in advance.

05

MODULE 5

ML foundations

The classical ML and model internals that still show up, and the reason they matter when you build LLM systems.

20 questions

Explain the bias variance trade off and why it still matters if you only work with LLMs.

What they are testing: *Fundamentals, and whether you can connect them to your actual job.*

Bias is error from a model too simple to represent the pattern. Variance is error from a model too sensitive to the specific sample it trained on. High bias underfits, so training and test error are both high. High variance overfits, so training error is low and test error is high. The target is the point of lowest total error, found with a validation set rather than by intuition.

The controls are familiar: more and better data, simpler models, L1 regularization for sparsity and feature selection, L2 to shrink weights while keeping all features, dropout in neural networks, early stopping, and cross validation.

The reason interviewers still ask is that the same failure reappears with new names in LLM work. Few shot examples tuned to your test cases overfit the prompt. A judge rubric tuned on fifty examples overfits to those fifty. Fine tuning on a small dataset memorizes it. Reporting eval scores on the same set you iterated against is train test leakage wearing a different hat. Answer the definition, then make that connection out loud.

Follow up they will push on: How do you know you are overfitting a prompt. A held out set that whoever wrote the prompt has never seen.

How do you handle an imbalanced dataset, and why is accuracy the wrong metric?

What they are testing: *Metric judgment, which is the transferable half of classical ML.*

With one percent positives, a model that always predicts negative is ninety nine percent accurate and completely useless. Accuracy hides the only cases you care about.

- ▶ Use precision, recall, F1, and PR AUC, which is more informative than ROC AUC under heavy imbalance.
- ▶ Choose between precision and recall by the cost of the error. Fraud screening and disease detection favor recall. Automated blocking and paid actions favor precision.
- ▶ Handle the imbalance with class weights, undersampling the majority, or oversampling with something like SMOTE, applied inside the training fold only, never before the split, or you leak.
- ▶ Tune the decision threshold on a validation set instead of accepting 0.5 by default. This is often the single highest leverage change.

Calibration deserves a mention because it comes up as a follow up. Two models with identical accuracy can have very different confidence quality. If you route on confidence, gate an abstention, or show a score to a user, you need calibrated probabilities. Check with a reliability curve and fix with Platt scaling or isotonic regression. In LLM systems this is what makes confidence based escalation trustworthy.

Follow up they will push on: Evaluate on the real distribution, never on resampled data, and use stratified splits.

QUESTION 83 · ML FOUNDATIONS

When would you use classical ML instead of an LLM?

What they are testing: *Engineering judgment. Reaching for an LLM for everything is a junior signal.*

Match the tool to the shape of the problem. Classical ML wins on structured tabular data with a clear label and reasonable volume. Gradient boosted trees such as XGBoost or LightGBM usually beat neural networks there, because tree ensembles handle mixed types, missing values, and non smooth decision boundaries with far less data and tuning. Classical ML also wins when you need millisecond latency, low unit cost at high volume, deterministic behavior, or an explanation you can defend to a regulator.

LLMs win on unstructured language, on tasks with little or no labeled data, on problems that need broad world knowledge, and on open ended generation where the specification is written in prose rather than a schema.

The strongest answer is usually hybrid. An LLM extracts structured fields from messy documents and a boosted tree scores them. Or a cheap classifier routes traffic and only the hard cases reach the LLM. You get language understanding where it is needed and predictable economics everywhere else.

Add the cost lens. At a million predictions a day, one model call per prediction is often the wrong architecture even when the quality is fine. Once the LLM has produced enough labeled examples, distill the task into a small model and keep the LLM for the tail.

Follow up they will push on: Why not a neural network on tabular data. Data efficiency, tuning burden, and interpretability.

QUESTION 84 · ML FOUNDATIONS

What is an embedding, and where do embeddings fail?

What they are testing: *Depth on the component every RAG system depends on.*

An embedding maps text into a vector space where similar meaning lands close together, learned by contrastive training on pairs that should be near each other and pairs that

should not. Similarity is usually cosine distance. That training objective explains the failure modes precisely, which is the part most candidates miss.

- ▶ Negation. Contract includes indemnity and contract excludes indemnity embed close together, because they share nearly all their content.
- ▶ Numbers, dates, and temporal reasoning. Vectors do not do arithmetic or ordering.
- ▶ Exact identifiers. SKUs, error codes, case numbers, and version strings need lexical matching.
- ▶ Domain jargon the embedding model never saw during training, which is common in legal, clinical, and internal company language.
- ▶ Long chunks, where one dominant topic washes out everything else in the vector.
- ▶ Asymmetry. A short query and a long passage come from different distributions, which is why query and passage prefixes or dedicated asymmetric models help.

The practical consequences: pair dense search with BM25, push dates and IDs into metadata filters instead of hoping the vector handles them, rerank with a cross encoder, keep chunks topically focused, and evaluate any embedding model on your own data before trusting a leaderboard. Changing embedding models means reindexing the whole corpus, so treat the index as versioned infrastructure and plan the migration.

Follow up they will push on: How would you test an embedding model. A labeled retrieval set from your own domain, measured with recall at k.

QUESTION 85 · ML FOUNDATIONS

When do you fine tune, and what are the options?

What they are testing: *Whether you know that fine tuning is usually the wrong first move, and that you know the landscape anyway.*

Decide by what is missing. Missing knowledge is a retrieval problem. Missing behavior, format, tone, or consistency on a narrow task is a fine tuning problem. Missing instructions is a prompting problem. The ladder is prompt engineering, then RAG, then fine tuning, because fine tuning hands you a training and evaluation pipeline you now own forever.

- ▶ Instruction tuning. Supervised fine tuning on instruction and response pairs. This is what turns a raw base model into one that follows directions.
- ▶ Full fine tuning. Updates every weight. Highest compute, highest risk of catastrophic forgetting, rarely justified for application work.
- ▶ LoRA. Freezes the base model and trains small low rank adapter matrices, cutting trainable parameters and memory by orders of magnitude with near parity quality on narrow tasks. Adapters can be swapped per customer, which is a real product advantage.
- ▶ QLoRA. LoRA on top of a quantized four bit base model, so training fits on a single smaller GPU. You trade some speed and precision for accessibility.

- Preference tuning. RLHF trains a reward model on human comparisons and optimizes the policy against it, which is powerful and operationally heavy. DPO skips the reward model and optimizes directly on preference pairs, which is far simpler and is the common default for teams without a research org.

For serving, quantization to int8 or four bit reduces memory and raises throughput with modest quality loss, and distillation, training a small student on a large teacher output, is often the better cost play than tuning the big model.

What the interviewer wants to hear: how much data you would need, usually one to ten thousand high quality examples with quality mattering far more than volume, a held out eval proving the tuned model beats the prompted baseline, and your plan for the day the base model version changes underneath you.

Follow up they will push on: How do you build the dataset. Production logs plus human correction, deduplicated, with a clean held out split.

QUESTION 86 • ML FOUNDATIONS

Explain how a transformer works at a high level, and why attention was the breakthrough.

What they are testing: *Whether you understand the architecture under everything you build on.*

A transformer processes a sequence of tokens by letting every token attend to every other token, building a context aware representation of each one. The core operation is self attention: for each token, the model computes how much to weight every other token through query, key, and value projections, then mixes their values by those weights. Stacking many attention and feed forward layers, with normalization and residual connections, produces the deep contextual representations that power generation.

Attention was the breakthrough because it solved two problems at once. Earlier recurrent models processed tokens in sequence, which made them slow to train and weak at connecting distant tokens, since information had to pass step by step through a bottleneck. Attention connects any two positions directly in one operation, so long range dependencies are captured, and because every position is computed in parallel rather than in sequence, training scales far better on modern hardware. That parallelism and scalability is what made training on internet scale data feasible.

For an application engineer the useful consequences are concrete: attention over the full context is why the model can use retrieved passages and conversation history, and its cost grows with sequence length, which is why long contexts are expensive and why trimming context helps latency and cost.

I would keep the depth appropriate to the role, the goal is to show I understand the mechanism, not to derive the math.

Follow up they will push on: Why does cost grow with context length. Attention relates tokens to other tokens, so the work scales super linearly with sequence length, which is the practical reason long prompts are costly.

QUESTION 87 · ML FOUNDATIONS

What is tokenization, and how does it cause real bugs?

What they are testing: *Whether you understand the layer between text and the model that quietly breaks things.*

A model does not see characters or words, it sees tokens, subword units produced by a tokenizer that splits text into common chunks. Words map to one or several tokens, and rare or unusual strings fragment into many. Everything about cost, context limits, and some failure modes traces back to this, because tokens, not characters, are the unit of billing and of the context window.

It causes concrete bugs. Character level tasks fail, counting letters, reversing a string, careful spelling, because the model sees tokens, not letters, so it has no direct view of the characters. Numbers and code tokenize awkwardly, which contributes to arithmetic mistakes. Different languages tokenize with very different efficiency, so the same text can cost far more tokens in one language than another, inflating cost and consuming context unevenly. Unusual inputs, long identifiers, unusual unicode, can fragment into surprisingly many tokens.

The practical implications are worth stating: estimate cost and context budget in tokens, not words, using the specific model tokenizer, expect and design around weak character level and arithmetic behavior rather than assuming the model can spell or compute, and be aware that non English or code heavy workloads carry a token overhead that changes the economics.

It is a small concept that explains a surprising share of odd behavior, which is why interviewers like it.

Follow up they will push on: Why does the model miscount the letters in a word. It sees subword tokens, not individual characters, so letter level operations are not something it can read off directly.

QUESTION 88 · ML FOUNDATIONS

How do you detect and fix overfitting and underfitting in practice?

What they are testing: *Whether you can diagnose the two failure modes from evidence, not just define them.*

Diagnose from the gap between training and validation performance. Underfitting shows as high error on both, the model is too simple or under trained to capture the pattern. Overfitting shows as low training error but high validation error, the model has memorized the training sample instead of learning the signal. The validation curve is the instrument, which is why an honest held out set is non negotiable.

For underfitting, add capacity or signal: a more expressive model, better features, longer training, or fewer constraints, since the problem is that the model cannot represent the relationship. For overfitting, do the opposite, reduce capacity or add constraint: more and more varied training data, which is usually the strongest fix, regularization, L1 or L2, dropout in neural networks, early stopping when validation error starts rising, and simpler architectures. Cross validation gives a more reliable estimate and guards against tuning to one lucky split.

The trap to name is leakage, letting information from validation or test bleed into training, resampling before the split, tuning on the test set, which makes a model look great and fail in production. It is the same failure that appears in LLM work when you report eval scores on the very examples you iterated against.

So the workflow is measure the gap, decide which failure you have, and apply the matching fix, rather than reaching for regularization reflexively.

Follow up they will push on: Training and validation error are both high and close. Which problem is it. Underfitting, so add capacity, features, or training, more data or regularization will not help a model that cannot represent the pattern.

QUESTION 89 · ML FOUNDATIONS

Explain precision, recall, F1, and ROC AUC, and when you use each.

What they are testing: *Whether you can pick the metric that matches the cost of being wrong.*

These describe classifier quality from different angles, and choosing the wrong one hides the failure that matters.

- ▶ Precision. Of the items predicted positive, how many really are. You optimize precision when a false positive is costly, blocking a legitimate transaction, flagging a good user, since a low precision system cries wolf.
- ▶ Recall. Of the items that are actually positive, how many you caught. You optimize recall when a false negative is costly, missing a disease, missing fraud, since a low recall system lets real cases through.
- ▶ F1. The harmonic mean of precision and recall, a single number when you need to balance the two, especially under class imbalance where accuracy is misleading.
- ▶ ROC AUC. The probability the model ranks a random positive above a random negative, across all thresholds, a threshold independent measure of ranking quality. Under heavy imbalance PR AUC is more informative, because ROC AUC can look strong while precision is poor.

The unifying idea is that precision and recall trade off against each other and you move between them by tuning the decision threshold, so the right operating point comes from the relative cost of the two error types, not from a default of 0.5. State which error you are willing to accept and set the threshold on a validation set to match.

Follow up they will push on: Your model has high ROC AUC but users see many false alarms. Why. Ranking is good but the threshold is loose, or precision is weak under imbalance, so tune the threshold and look at PR AUC.

QUESTION 90 · ML FOUNDATIONS

What is cross validation, and how does data leakage sabotage evaluation?

What they are testing: *Whether you can produce an honest performance estimate, the foundation of all evaluation.*

Cross validation estimates how a model generalizes by splitting the data into folds, training on some and validating on the held out fold, and rotating so every point is validated once. Averaging across folds gives a more stable estimate than a single split and reduces the chance that one lucky or unlucky partition misleads you, which matters most when data is limited.

Leakage is when information that would not be available at prediction time sneaks into training, and it inflates your estimate so a model looks excellent in evaluation and disappoints in production. It is the most common way teams fool themselves.

- ▶ Preprocessing before the split. Fitting scaling, imputation, or resampling like SMOTE on the full dataset leaks test statistics into training, so all such steps go inside the fold.
- ▶ Temporal leakage. Using future data to predict the past, so time series need splits that respect chronology, not random folds.
- ▶ Group leakage. The same entity in both train and test, the same patient, user, or document, so split by group when records are correlated.
- ▶ Target leakage. A feature that encodes the outcome, available only because the event already happened.

The discipline is to decide exactly what is known at prediction time and let nothing else into training, and in LLM work the same rule means holding out eval examples that whoever tuned the prompt never saw.

Follow up they will push on: Why apply SMOTE inside the fold and not before. Resampling before the split lets synthetic points derived from test rows leak into training, inflating the score, so it must happen only on the training fold.

How does a model actually learn? Explain gradient descent and loss at a high level.

What they are testing: *Whether you understand the training loop conceptually, not just that training happens.*

Learning is minimizing a loss function, a number that measures how wrong the model predictions are on the training data. Training adjusts the model parameters to make that number smaller, and the method for doing so is gradient descent.

The gradient is the direction in which the loss increases fastest with respect to the parameters, computed efficiently by backpropagation, which applies the chain rule to attribute the error back through the network. Gradient descent takes a small step in the opposite direction, downhill, and repeats, so the model iteratively improves. The step size is the learning rate, too large and training diverges or oscillates, too small and it crawls. In practice we use stochastic or mini batch gradient descent, estimating the gradient on small batches rather than the whole dataset, which is faster and scales.

A few consequences are worth knowing even for application work. The loss landscape is not a simple bowl, so training can stall or land in poor regions, which is why learning rate schedules and optimizers matter. And the same objective that fits the training data can overfit it, which is why we watch validation loss and stop when it stops improving.

The level to aim for is showing I understand that training is guided error reduction by following gradients, not reciting optimizer internals unless the role calls for it.

Follow up they will push on: What does the learning rate control. The size of each downhill step, too high diverges, too low is slow, which is why it is one of the most important hyperparameters to tune.

What is transfer learning, and why is it the foundation of modern AI?

What they are testing: *Whether you understand why pretrained models changed the economics of building.*

Transfer learning is reusing a model trained on one large task as the starting point for another, rather than training from scratch. A model pretrained on a huge general corpus has already learned broadly useful representations, and you adapt those to your specific task with far less data and compute than starting from nothing.

This is the foundation of modern AI because it changed the economics. Pretraining a large model is enormously expensive and needs massive data, which almost no application team can do. Transfer learning lets everyone build on top of that once paid cost: the pretrained model brings general language or visual understanding, and

adaptation, fine tuning, or even just prompting, specializes it cheaply. It is why a small team can ship a capable system without a research budget.

The adaptation spectrum runs from heaviest to lightest: full fine tuning updates all weights, parameter efficient methods like LoRA update a small fraction, and in context learning, prompting and few shot examples, adapts behavior with no weight updates at all. Foundation models pushed this so far that for many tasks no training is needed, you just prompt.

The practical framing for an interview is that using an LLM at all is transfer learning, you are standing on a pretrained model and adapting it, and the question is only how heavy an adaptation the task actually needs.

Follow up they will push on: How does in context learning relate to transfer learning. It is the lightest form, the model adapts to your task from examples in the prompt with no weight change, which is why prompting often replaces fine tuning.

QUESTION 93 • ML FOUNDATIONS

Does feature engineering still matter in the age of deep learning and LLMs?

What they are testing: *Whether you have a nuanced view rather than a slogan in either direction.*

The honest answer is that it depends on the data type, and dismissing feature engineering entirely is a mistake. Deep learning did reduce the need for manual feature crafting on unstructured data, images, audio, text, where the network learns useful representations directly, which is a real shift from classical pipelines that leaned heavily on hand designed features.

But on structured tabular data, which is most of what businesses run on, feature engineering remains one of the highest leverage activities. Domain informed features, ratios, aggregations, time based features, encodings of categorical variables, often move performance more than swapping models, and gradient boosted trees plus good features still beat neural networks on many tabular problems. So the discipline did not disappear, it concentrated where it pays.

In LLM systems the idea reappears under different names. Context construction, what you retrieve and how you format it, is effectively feature engineering for the prompt. Deciding which fields to extract, how to structure the input, and what metadata to attach is the same skill of shaping raw data into a form the model can use well.

So the mature position is that representation still matters, the model learns some of it for you on unstructured data, and you still earn a lot by shaping structured data and context deliberately.

Follow up they will push on: Where is feature engineering hiding in an LLM app. In context construction and data extraction, choosing and formatting what goes into the prompt is the same work as crafting features.

What are logits, softmax, and probabilities, and how do they relate to sampling?

What they are testing: *Whether you understand what the model actually outputs before text appears.*

At each step a language model outputs a vector of logits, one raw score per token in the vocabulary. Logits are unbounded and not yet probabilities. Softmax converts them into a probability distribution by exponentiating and normalizing, so the values are positive and sum to one, giving the model estimated probability of each possible next token.

Sampling then selects a token from that distribution, and the decoding controls operate right here. Temperature scales the logits before softmax, low temperature sharpens the distribution toward the highest probability tokens, high temperature flattens it toward variety. Top p keeps the smallest set of tokens whose cumulative probability passes p, and top k keeps a fixed count, both truncating the tail before sampling. Greedy decoding just takes the most probable token.

Understanding this chain explains real behavior. It is why temperature and top p are the levers for determinism versus creativity, and why they act on the same underlying distribution so you tune one, not all at once. It also underpins confidence signals, the probability of the chosen tokens can inform how certain the model is, though it must be treated cautiously since these probabilities are not perfectly calibrated.

The takeaway for an engineer is that the model emits a distribution, not a word, and your sampling settings decide how you draw from it.

Follow up they will push on: Is a high token probability the same as the answer being correct. No, it reflects the model confidence in the next token given its training, which can be confidently wrong, so probabilities are a weak, uncalibrated signal to use with care.

Explain model compression: quantization, pruning, and distillation.

What they are testing: *Whether you know how to make models cheaper and faster to serve.*

These make a model smaller, faster, or cheaper to run, which matters because a capable model can be too slow or too expensive to serve at scale. They work differently and trade quality against efficiency in different ways.

- **Quantization.** Represent weights and sometimes activations at lower numerical precision, sixteen or eight bit or four bit instead of full precision, which cuts memory and speeds inference with usually modest quality loss. It is the most common and highest leverage lever, and QLoRA even trains on top of a quantized base to fit a single smaller GPU.

- ▶ **Pruning.** Remove weights or structures that contribute little, shrinking the model. Unstructured pruning zeros individual weights, structured pruning removes whole units for real speedups, at the risk of quality loss that may need fine tuning to recover.
- ▶ **Distillation.** Train a small student model to mimic a large teacher output, transferring much of its capability into a far cheaper model. This is often the best cost play, and it pairs naturally with using a big model to generate labels and then distilling the task.

The framing to give is that these are how you move a research quality model into production economics: quantize first for an easy win, distill when you have a narrow task and want a small dedicated model, and prune when you need to squeeze a specific deployment. All of them are validated against your eval set, since the point is efficiency without an unacceptable quality drop.

Follow up they will push on: You need lower latency and cost on a narrow task. What do you reach for. Distill a small model on the large model outputs, or quantize, since a focused small model can match the big one on a narrow task at a fraction of the cost.

QUESTION 96 • ML FOUNDATIONS

Explain RLHF and DPO. How do models get aligned to human preferences?

What they are testing: *Whether you understand the alignment step that turns a capable model into a useful one.*

A base model trained only to predict the next token is capable but not aligned, it does not reliably follow instructions or match human preferences about helpfulness and safety. Preference tuning is the step that fixes this, teaching the model what humans prefer among possible responses, and it typically follows supervised instruction tuning.

RLHF, reinforcement learning from human feedback, does this in stages. Humans compare pairs of model responses, those comparisons train a reward model that scores how preferred a response is, and then the policy is optimized with reinforcement learning to produce responses the reward model rates highly, usually with a constraint to avoid drifting too far from the original model. It is powerful but operationally heavy, three models and an RL loop, with its own failure mode of reward hacking, where the model games the reward model rather than genuinely improving.

DPO, direct preference optimization, reaches a similar goal more simply. It skips the separate reward model and RL loop and optimizes the model directly on the preference pairs with a single training objective, which is far easier to run and has become a common default for teams without a research organization.

For an application engineer the relevance is that this is why hosted models follow instructions and refuse harmful requests, and that you can apply the same preference tuning, more often DPO, to specialize a model behavior when prompting is not enough.

Follow up they will push on: What is reward hacking. The policy learns to maximize the reward model score in ways that do not reflect real quality, exploiting flaws in the proxy, which is a core risk of optimizing against a learned reward.

QUESTION 97 · ML FOUNDATIONS

How do models handle position and long context, and why does the middle get lost?

What they are testing: *Whether you understand why context length is not free and long context has quality caveats.*

Self attention itself has no inherent sense of order, so transformers add positional information to tell the model where each token sits. Modern models use schemes, rotary and related relative position methods, that encode relative distance and extend to longer sequences better than the earliest fixed encodings, which is part of how context windows grew so large.

But a large window does not mean uniform attention across it. Models empirically recall information best at the beginning and end of the context and worst in the middle, the lost in the middle effect, so a critical fact buried in the center of a huge prompt is used less reliably than the same fact near the edges. More tokens is not more usable signal.

This has direct engineering consequences. It is a reason to retrieve and curate a tight context rather than stuffing everything in, to place the most important material where the model attends best rather than at random depth, and to be skeptical of claims that you can dump a whole corpus into a long window and skip retrieval. It also reinforces why cost and latency grow with length, since attention relates tokens across the whole sequence.

So the practical stance is that long context is a useful tool for giving the model room, not a license to ignore what you put where.

Follow up they will push on: Why not just put everything in a long context and skip RAG. Cost and latency grow with length and recall sags in the middle, so a curated context of the right passages beats an undifferentiated dump.

QUESTION 98 · ML FOUNDATIONS

What is perplexity, and why is it a poor measure of whether a task succeeded?

What they are testing: *Whether you know the difference between a language modeling metric and a task metric.*

Perplexity measures how well a model predicts a piece of text, essentially how surprised it is by the actual next tokens, lower meaning the text is more predictable to the model. It is a natural intrinsic metric for language modeling and useful for

comparing models on the same corpus or tracking pretraining, because it directly reflects the training objective.

It is a poor task metric because predicting text well is not the same as doing the job. A model can assign high probability to fluent, plausible text that is factually wrong, unhelpful, or misses the user intent, so low perplexity does not imply a correct answer, a successful retrieval, or a satisfied user. It measures fluency and fit to a distribution, not truth or usefulness, and it is not even defined for the outcomes you actually care about, like task completion or groundedness.

This sits alongside the other intrinsic metrics that mislead on generation. BLEU and ROUGE measure n gram overlap with a reference and correlate poorly with quality on open ended answers, since there are many good answers that share few tokens with any one reference.

The correct move is to evaluate against task defined outcomes, task success, groundedness, retrieval recall, human or calibrated judge ratings, and treat perplexity as a model comparison tool, not evidence that a feature works.

Follow up they will push on: Your model has lower perplexity but users prefer the other one. How is that possible. Perplexity rewards predicting text, not being helpful or correct, so a model can fit the distribution better yet produce less useful answers, which is why you measure task outcomes.

QUESTION 99 · ML FOUNDATIONS

Cosine, dot product, or Euclidean distance for vector similarity. How do you choose?

What they are testing: *Whether you understand the metric under every similarity search you run.*

These define what near means in the vector space, and the right choice depends on the embedding model and whether magnitude carries meaning.

- ▶ Cosine similarity. Measures the angle between vectors and ignores their magnitude, so it compares direction, which is meaning, regardless of length. It is the most common default for text embeddings because semantic similarity is about direction, not how long the vector is.
- ▶ Dot product. Combines direction and magnitude. When embeddings are normalized to unit length, dot product and cosine are equivalent, which is why many systems normalize and use dot product for efficiency. When magnitude is meaningful, dot product reflects it, for better or worse.
- ▶ Euclidean distance. Straight line distance, sensitive to magnitude, common in some settings but less typical for normalized text embeddings, where it tends to agree with cosine after normalization anyway.

The practical rule is to use the metric the embedding model was trained with, because the model learned its geometry under a specific objective and mismatching the metric

silently degrades retrieval. Most text embedding models expect cosine or, equivalently, normalized dot product. Normalizing vectors makes the choice largely moot between cosine and dot product and is a common convention.

So the answer is not a favorite metric, it is match the metric to the model, normalize, and confirm on your own retrieval eval rather than assuming.

Follow up they will push on: When are cosine and dot product the same thing. When vectors are normalized to unit length, magnitude is fixed, so dot product reduces to cosine, which is why normalization is a common default.

QUESTION 100 · ML FOUNDATIONS

What is regularization, and how do L1, L2, and dropout actually prevent overfitting?

What they are testing: *Whether you understand the concrete mechanisms that keep a model from memorizing its training data.*

Regularization is any technique that discourages a model from fitting the training data too closely, trading a little training accuracy for better performance on data it has not seen. It matters because an unconstrained model with enough capacity will memorize noise, and the whole point of the model is to generalize, not to recite.

- ▶ L2, weight decay. Add the sum of squared weights to the loss, which pushes weights toward small values. Large weights mean the model leans hard on a few features, so shrinking them spreads reliance and produces a smoother function that generalizes better.
- ▶ L1. Add the sum of absolute weights, which drives some weights exactly to zero. That performs feature selection as a side effect, yielding a sparse model that ignores inputs it does not need.
- ▶ Dropout. During training, randomly zero a fraction of neurons each pass, so the network cannot depend on any single unit and must learn redundant, robust features. At inference you use the full network, which behaves like averaging over many thinner networks.

The unifying idea is a penalty on complexity, whether that penalty is on weight magnitude or on co dependence between neurons. Related tools do the same job from other angles: early stopping halts training before it starts fitting noise, and more data is the strongest regularizer of all because it makes memorization harder than learning the real pattern.

The trade off is set by strength. Too little and the model overfits, too much and it underfits because you have constrained it into a model too simple to capture the signal. You choose the strength the same way you choose any hyperparameter, by watching the gap between training and validation performance on a held out set and tuning until validation error is lowest. The failure mode is tuning regularization against the test set, which quietly leaks it and leaves you with a number that does not hold up in production.

Follow up they will push on: You increase L2 and both training and validation accuracy fall. What does that tell you. You have over regularized into underfitting, the model is now too constrained to fit the signal, so dial the penalty back.

You are not behind. You are unspecialized.

Knowing the answers is the first half. Having projects that prove you have built these systems is what gets the callback.

skool.com/baswe-ai

baswe.**Ai** Engineer Accelerator™